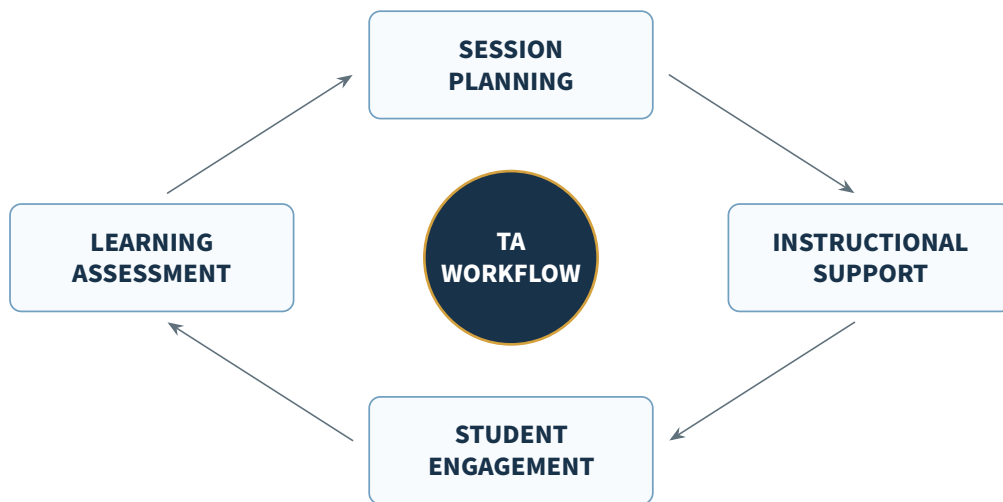


Operating Systems

Teaching Assistant Guide



COURSE OUTLINE

A lab-aligned guide to Linux system interfaces, process management, CPU scheduling, synchronization, deadlocks, memory management, virtual memory, and file-system organization.

PREPARED BY

Reeyan Afzal Khan

Graduate Research Assistant

Learning Path

Course Map: From a Program to a Complete Operating System	ii
Stage 1 Operating-System Foundations	1
Chapter 1 Operating Systems at a Glance	2
Chapter 2 Linux Processes, Files, and System Interfaces	9
Lab 1 of 10 — Directories and process creation	14
Lab 2 of 10 — File scanning and directory utilities	16
Stage 2 Shell, Processes, and CPU Management	19
Chapter 3 Shell Scripting and Command-Line Automation	20
Lab 3 of 10 — Bash scripting fundamentals	24
Chapter 4 CPU Scheduling	26
Lab 4 of 10 — CPU scheduling algorithms	33
Stage 3 Concurrency and Resource Management	35
Chapter 5 Interprocess Communication and Synchronization	36
Lab 5 of 10 — Synchronization and shared memory	40
Chapter 6 Threads, Resource Allocation, and Deadlocks	41
Lab 6 of 10 — Threads, Banker’s algorithm, and deadlock reasoning	45
Stage 4 Memory Management	46
Chapter 7 Contiguous Memory Allocation	47
Lab 7 of 10 — First fit, best fit, and worst fit	51
Chapter 8 Virtual Memory and Page Replacement	52
Lab 8 of 10 — Mutexes and page-replacement policies	57
Stage 5 File Systems and Storage	58
Chapter 9 Directories and File-System Namespace	59
Lab 9 of 10 — Directory-structure visualization	62
Chapter 10 File Organization and Disk-Block Allocation	63
Lab 10 of 10 — File records and indexed allocation	67
Revision Map and Capstone Practice	68

Course Map: From a Program to a Complete Operating System

An operating system sits between application programs and hardware. The course becomes easier when every topic is attached to one of four questions: *what runs*, *who gets the CPU*, *where does data live in memory*, and *how does persistent data reach storage*?

1. Interface to the OS — Linux commands, files, directories, processes, `fork()`, and basic system interfaces.



2. CPU and concurrency — scheduling, process interaction, semaphores, shared memory, POSIX threads, mutexes, and deadlock.



3. Memory — contiguous allocation, fragmentation, virtual memory, page faults, and replacement policies.



4. Persistent storage — directory trees, file records, file-system metadata, and indexed allocation.

Core path

Use the labs as checkpoints rather than isolated programs. Each one exposes a real OS abstraction: a process identifier, an open directory stream, a scheduling queue, a semaphore count, a safe-state test, a memory hole, a page frame, or an indexed block.

Lab	Practical focus	Core OS idea
1	Directories and <code>fork()</code>	OS services, processes, PIDs
2	File text processing and directory reading	File I/O and directory APIs
3	Bash scripting	Command interpreter and automation
4	FCFS, SJF, Priority, Round Robin	CPU scheduling
5	Producer-consumer and shared memory	Synchronization and IPC
6	Banker's algorithm, deadlock detection, threads	Resource safety and concurrency
7	First, best, and worst fit	Contiguous memory allocation
8	Mutexes, FIFO, LRU, LFU	Mutual exclusion and page replacement
9	Directory visualization	Hierarchical file-system namespace
10	Records and indexed allocation	File organization and block allocation

Key idea

Do not memorize the algorithms as unrelated loops. For every algorithm, ask: **what state does the OS maintain, what event changes that state, and what policy decides the next action?**

Stage 1

Operating-System Foundations

Operating Systems at a Glance

Chapter at a glance

Before you start: You should be comfortable with basic C programs and the idea that a computer has a CPU, memory, and storage.

By the end, you can: Explain why an OS exists, distinguish kernel and user mode, classify OS services, and connect the main course topics into one execution story.

1.1 Start with something you have already done

Open a terminal and type `ls`. A list of filenames appears. Nothing about that is obvious once you look closely:

- Your shell was running. It had to *stop* doing that and start a different program.
- `ls` needed memory — but so does every other program currently running, and none of them interfered.
- The filenames live as magnetic or flash state on a disk. Something turned that into text.
- The output reached your screen, which `ls` does not own and cannot address directly.

Four different subsystems cooperated, and you typed two letters. The **operating system (OS)** is the software that made all four happen.

Key idea

An OS does two jobs at once. It is an *abstraction layer*, turning awkward hardware into workable objects like processes, files, and memory addresses. It is also a *resource manager*, deciding who gets what, for how long, and with what permission.

Nearly every design decision in this course comes from those two jobs pulling in different directions.

1.2 The layers involved

Your program sits at the top. The hardware sits at the bottom. Everything between is the OS's responsibility.

Applications: shell, editor, browser, compiler, user programs

System-call interface and libraries

Kernel: processes, scheduler, memory manager, file systems, device drivers

Hardware: CPU, RAM, storage, timers, network and I/O devices

Read the diagram as a rule about *reach*. An application cannot skip a layer. It cannot address the disk

controller, and it cannot read another process's memory. Every downward step goes through the interface directly beneath it.

1.3 Kernel mode and user mode

That restriction is enforced by the CPU itself, not by good manners.

Modern processors run in at least two privilege levels. Your programs run in **user mode**, where privileged instructions and direct hardware access are simply unavailable — attempting them traps instead of executing. The kernel runs in **kernel mode**, where everything is permitted.

Control crosses that boundary in exactly three ways, and it is worth memorising them now because the rest of the course keeps returning to them:

System call The program *asks*. It wants something only the kernel can do.

Exception The current instruction *cannot proceed* — divide by zero, bad memory access, page fault.

Interrupt Hardware *demands attention* — a timer tick, a keypress, a disk finishing a read.

Plain idea

User mode is a blast wall. A buggy program should be free to crash itself, and unable to take down the machine, corrupt another process's memory, or reprogram a disk controller on its way out. This is why an infinite loop in your code does not freeze the whole computer. The timer interrupt fires regardless of what your program is doing, and control returns to the kernel whether your program likes it or not.

1.4 System calls are the operating-system API

A **system call** is a request for a service that needs kernel authority. They group into a handful of families:

Family	What it does	Seen in this course
Process control	Create, terminate, wait for, or replace a running program	<code>fork()</code> , <code>waitpid()</code> in Lab 1
File management	Open, read, write, seek, close, inspect metadata	<code>fopen()</code> , <code>fgetc()</code> in Lab 2
Device management	Read and write devices, configure device state	Chapters 9–10
Information	Process identifiers, time, resource limits	<code>getpid()</code> in Lab 1
Communication	Pipes, shared memory, message queues, sockets, signals	Lab 5
Protection	Permissions, identities, access checks	Chapter 2

Common mistake

`printf()` is not a system call. It is a C library function that formats your text and then calls `write()`, which is the system call.

The distinction matters more than it sounds. The library buffers output, so `printf()` may return long before anything reaches the screen. In Lab 1 you fork a process, and both parent and child inherit a copy of that buffer — which is how a single `printf()` can produce output twice. Add `fflush(stdout)` before `fork()` and the duplicate disappears.

1.5 One command, every subsystem

Return to `ls`, and trace it properly. The shell creates a process. The scheduler decides when it runs. Virtual memory hands it an address space that appears to start at zero regardless of where it physically sits. The file system turns the pathname into a location. The storage stack reads blocks. An interrupt announces the read finished. The terminal driver receives the output.

Key idea

The topics in this course are not a list of separate features. They are cooperating subsystems, and almost every non-trivial program execution needs all of them at once. Each later chapter zooms in on one of the steps above.

1.6 Program, process, and thread

These three words get used loosely in conversation and precisely in operating systems. The difference is worth getting right early.

A **program** is passive: instructions and data sitting in a file. It does nothing. A **process** is a running instance of a program, with state — registers, program counter, address space, open files, scheduling information, credentials. A **thread** is a single flow of execution inside a process.

Analogy

A program is a recipe. A process is one cook actually working from it, with their own bench space, ingredients, and half-finished pan. A thread is a pair of hands.

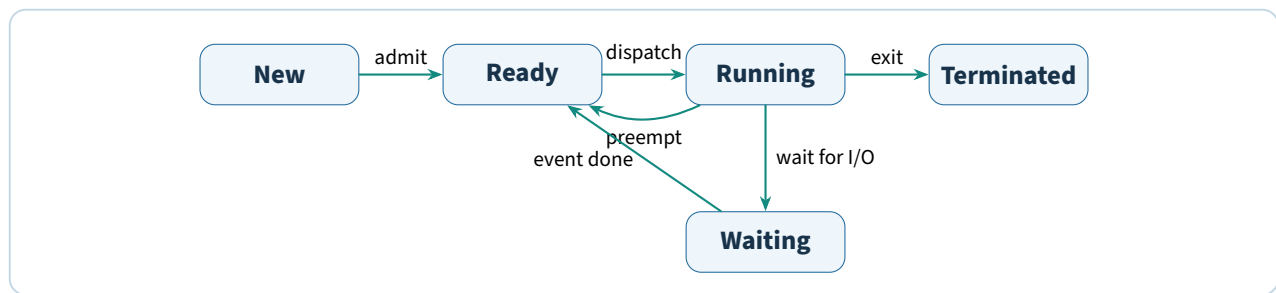
Two cooks working from the same recipe do not share a bench — that is two processes. Two pairs of hands belonging to one cook share everything on that bench — that is two threads, and it is exactly why threads need locks and processes usually do not.

Concept	Owns / represents	Typical question
Program	Executable instructions and static data	What code could run?
Process	Resource container plus execution state	Which running instance is this?
Thread	Schedulable execution flow	Which sequence of instructions runs now?

Running the same program twice gives two processes. They have different process IDs, separate memory, and no ability to see each other's variables. You will observe this directly in Lab 1: after `fork()`, parent and child both continue from the same line with the same variables — and changing a variable in one has no effect on the other.

1.7 The process-state model

A process is not always running. Most of the time it is waiting for something. The standard five-state model captures that:



Two arrows deserve attention, because students routinely confuse them.

Running to Ready is **preemption**. The process was perfectly able to continue; the OS stopped it anyway, usually because its time slice expired. This is the arrow Round Robin in Chapter 4 depends on.

Running to Waiting is **blocking**. The process cannot continue — it asked for data from disk and the data is not there yet. No scheduling policy can help; only the event can.

Common mistake

“Ready” does not mean idle. A ready process wants the CPU right now and cannot have it, because another process is using it. On a busy machine, the ready queue is where processes spend most of their existence.

1.8 The central problem: controlled sharing

An operating system exists because a computer has a small number of physical resources that many programs want to use at the same time. The OS therefore solves two problems simultaneously:

1. **Abstraction:** present hardware through easier objects such as processes, files, virtual memory, and sockets.
2. **Arbitration and protection:** decide who gets each resource, for how long, and under what permissions.

These goals sometimes conflict. Maximum utilization might favor long uninterrupted CPU runs, while interactive responsiveness favors frequent preemption. Strong isolation may add overhead. Much of operating-system design is therefore about carefully chosen trade-offs.

Analogy

Think of the OS as the management layer of a large shared building. Applications are tenants. The CPU is a limited set of work rooms, memory is rented floor space, storage is the archive, and devices are shared facilities. The OS creates usable rules and identities, schedules access, and prevents one tenant from walking into another tenant’s locked room.

1.9 Mechanism versus policy

A useful systems distinction is between **mechanism** and **policy**.

- A **mechanism** provides a capability: a timer interrupt can stop a running process; a page table can translate addresses; a mutex can provide mutual exclusion.
- A **policy** decides how to use that capability: Round Robin chooses when a timer should cause a scheduling change; a page-replacement policy chooses a victim; a lock protocol decides which shared state must be protected.

Separating policy from mechanism makes systems easier to change. The same context-switch mechanism can support FCFS, priority scheduling, or Round Robin.

1.10 How control enters the kernel

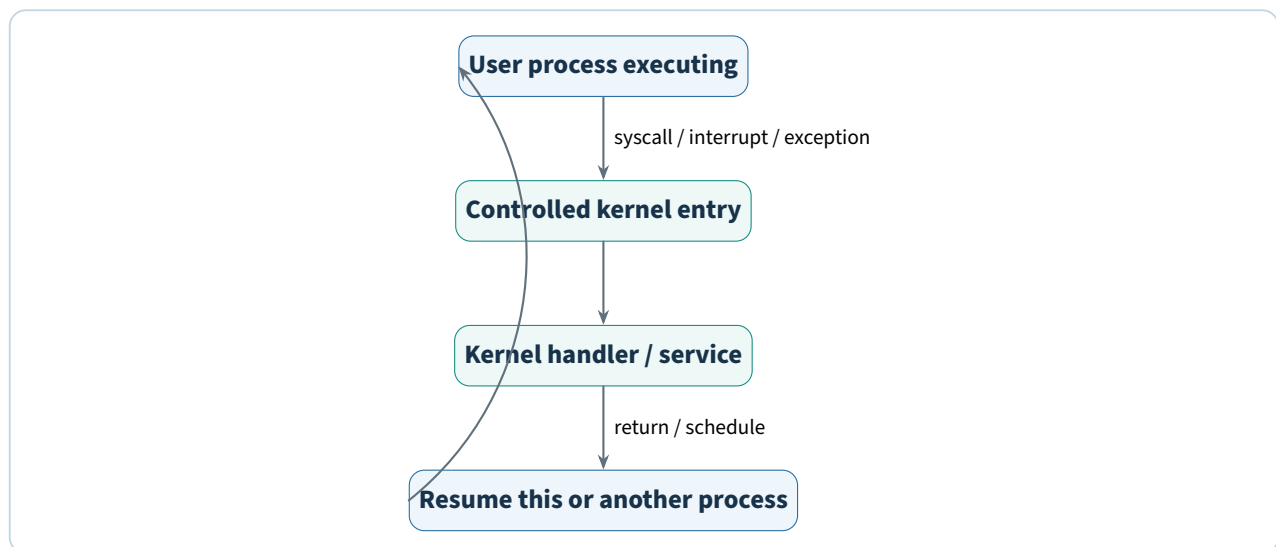
The CPU does not run the kernel continuously. Most of the time a user process is executing. Kernel code runs when control is transferred for a reason:

System call A program deliberately requests an OS service, for example `read()` or `fork()`.

Exception / fault The current instruction cannot proceed normally, for example division by zero, invalid memory access, or a page fault.

Interrupt Hardware asynchronously requests attention, for example a timer tick, keyboard input, or disk-I/O completion.

The kernel saves enough processor state to later resume execution, determines the cause, runs the appropriate handler, and finally returns to a user process selected by the scheduler.



1.11 What the kernel must remember

To suspend one process and later continue it correctly, the OS needs a data structure commonly called a **process control block (PCB)**. Exact fields vary, but conceptually it stores:

- process identifier and current state;
- saved CPU registers and program counter;
- scheduling parameters and accounting information;
- memory-management references such as page-table information;
- open files and I/O state;
- credentials and protection information.

A **context switch** saves the execution context of one thread/process and restores another. Context switching enables multiprogramming but does not directly perform useful application work, so excessive switching is overhead.

1.12 Major operating-system design families

OS kernels can be organized in different ways.

Style	Basic idea	Trade-off
Monolithic kernel	Most core services run in one privileged kernel address space	Fast internal calls; a bug in privileged code can have broad impact
Layered / modular	Kernel functionality is separated into clearer modules/layers	Easier organization and maintenance; boundaries can add complexity
Microkernel	Keep a small privileged core and move more services to user-space servers	Strong isolation and modularity; extra communication may cost performance
Hybrid	Mix techniques to balance performance and structure	Practical compromise used by many modern systems

Beyond the core

Virtual machines and containers. Virtualization adds another abstraction boundary. A hypervisor can present virtual hardware to guest operating systems, while containers generally share one host kernel but isolate process namespaces, filesystems, and resources. These ideas reuse the same OS principles of abstraction, scheduling, protection, and accounting.

Check your understanding

1. Is a timer interrupt a policy or a mechanism? What scheduling policy can use it?
2. Why is a context switch necessary when the CPU changes from one process to another?
3. Give one example each of a system call, an exception, and an interrupt.

Check your understanding

1. Why does an application need a system call to create another process?
2. What do threads in one process share, and what stays private to each?
3. Name one event that moves a running process into the waiting state.
4. A process sits in Ready for 200 ms. Is it blocked? What is it waiting for?

Answers.

1. Creating a process means allocating an address space, a process-table entry, and an identity. All of those are kernel-owned structures, and letting user code write them directly would remove every protection guarantee the OS makes.
2. Shared: address space, open-file table, code, globals, and the heap. Private: registers, program counter, and stack. Sharing the heap is exactly why two threads writing one variable need a lock.
3. A read from disk, waiting on a child with `waitpid()`, or blocking on a semaphore — anything where the process cannot proceed until an external event occurs.
4. Not blocked. Ready means it is able to run and is waiting only for a turn on the CPU. A blocked process waits for an event; a ready one waits for the scheduler.

Chapter recap

- The OS provides abstractions while managing and protecting hardware resources; those two goals frequently conflict.

- The CPU enforces the user/kernel boundary. Control crosses it by system call, exception, or interrupt — and by nothing else.
- A process is a running instance with its own resources; a thread is a flow of execution that shares them.
- Preemption and blocking both stop a running process, for completely different reasons.
- Scheduling, memory, I/O, and file systems all participate in ordinary program execution.

Linux Processes, Files, and System Interfaces

Chapter at a glance

Before you start: Read Chapter 1 first.

By the end, you can: Use process identifiers and `fork()`, explain directory streams and file I/O, distinguish standard C library calls from kernel system calls, and diagnose common resource-management mistakes.

2.1 The Unix idea: many resources look like files

Unix-like systems expose regular files, terminals, pipes, device endpoints, and many kernel objects through file-descriptor based interfaces. At the C library level, a `FILE *` stream adds buffering and formatted I/O. At the lower POSIX level, integer file descriptors are used by calls such as `open()`, `read()`, `write()`, and `close()`.

Layer	Examples	Main idea
C standard I/O	<code>fopen</code> , <code>fgetc</code> , <code>fgets</code> , <code>fprintf</code> , <code>fclose</code>	Buffered, portable stream interface
POSIX file API	<code>open</code> , <code>read</code> , <code>write</code> , <code>close</code>	File-descriptor oriented OS interface
Directory API	<code>opendir</code> , <code>readdir</code> , <code>closedir</code>	Traverse directory entries
Process API	<code>fork</code> , <code>getpid</code> , <code>wait</code> , <code>exec</code> family	Create and manage processes

2.2 Reading a directory

`opendir()` opens a directory stream and returns a `DIR *`. Repeated calls to `readdir()` return pointers to `struct dirent` entries until no entries remain. `closedir()` releases the stream.

```
DIR *dir = opendir(path);
if (dir == NULL) {
    perror("opendir");
    return 1;
}

struct dirent *entry;
while ((entry = readdir(dir)) != NULL) {
    printf("%s\n", entry->d_name);
}
closedir(dir);
```

Common mistake

Closing the directory *inside* the `while (readdir(...))` loop invalidates the stream after the first entry. Resource release belongs after traversal unless the algorithm intentionally stops early.

2.3 Process creation with fork

On Unix-like systems, `fork()` creates a child process by duplicating the calling process's execution context. Both processes return from the same call, but with different return values:

- negative: process creation failed;
- zero: currently executing in the child;
- positive: currently executing in the parent, and the value is the child's PID.

```
pid_t pid = fork();
if (pid < 0) {
    perror("fork");
    return 1;
} else if (pid == 0) {
    printf("child PID = %ld\n", (long)getpid());
} else {
    printf("parent PID = %ld, child PID = %ld\n",
        (long)getpid(), (long)pid);
}
```

Plain idea

After `fork()`, there are two independently scheduled processes continuing from nearly the same instruction. Therefore output order is not guaranteed unless explicit synchronization is added.

2.4 File scanning as an OS exercise

The supplied labs count spaces and implement a miniature `grep`. These examples matter because they combine pathname resolution, file opening, repeated reads, string processing, and clean closure.

A robust loop should test the read operation directly rather than using `feof()` as the loop condition:

```
int ch;
while ((ch = fgetc(fp)) != EOF) {
    if (ch == ' ') spaces++;
}
```

Common mistake

`feof(fp)` becomes true only *after* a read attempts to pass end-of-file. A `while (!feof(fp))` pattern often processes one invalid or stale value at the end.

2.5 Resource lifetime discipline

Operating-system programming is full of acquire-use-release pairs:

- `fopen()` / `fclose()`
- `opendir()` / `closedir()`
- `shmat()` / `shmdt()`
- `pthread_mutex_init()` / `pthread_mutex_destroy()`

A reliable program checks the acquire step, avoids using invalid handles, and releases resources along every relevant control path.

Check your understanding

1. What does a positive return value from `fork()` mean in the parent?
2. Why is `while (!feof(fp))` usually weaker than checking the result of `fgetc()` or `fgets()` directly?
3. Where should `closedir()` normally appear relative to a `readdir()` loop?

2.6 A process is more than a PID

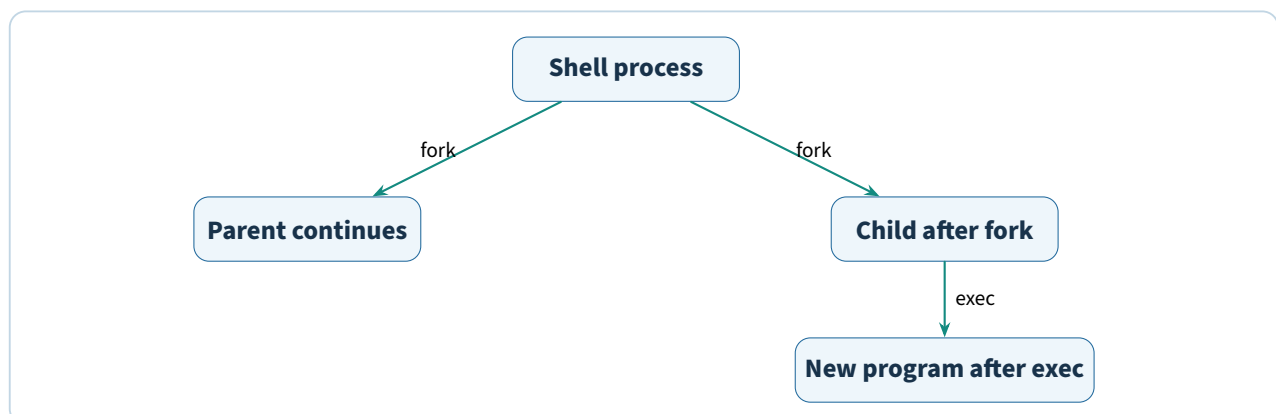
The process identifier is only a label. The meaningful idea is that each process has an execution context and a protected resource environment. Two instances of the same executable can therefore have different PIDs, stacks, heaps, open files, current directories, and scheduling states.

2.6.1 Parent-child process relationships

On Unix-like systems, processes form a hierarchy. A process created by `fork()` has a parent process. Immediately after a successful `fork`, parent and child begin with nearly identical logical memory contents, but they are separate processes. Modern systems normally implement this efficiently with **copy-on-write**: physical pages may initially be shared read-only and copied only when one process modifies a page.

Key idea

`fork()` creates a new process; `exec()` replaces the current process image with a new program. Shells usually use them together: `fork` a child, let the child `exec` the requested command, and let the parent shell wait or continue depending on whether the job is foreground or background.



2.7 Waiting, zombies, and orphans

When a child terminates, the kernel retains a small amount of exit information until its parent collects it using a `wait` operation. A terminated child whose status has not yet been collected is commonly called a **zombie**. A child whose parent exits first becomes an **orphan** and is adopted/re-parented according to the operating system's process-management rules.

Common mistake

A zombie is not a running process consuming CPU. It is a terminated process with an unrealed process-table entry. The resource leak is the retained bookkeeping entry, not continued execution.

2.8 File descriptors: per-process references to open objects

A process usually has a table of small integer **file descriptors**. By convention, descriptors 0, 1, and 2 start as standard input, standard output, and standard error. An entry points to kernel-managed open-file state such as the current file offset and access mode.

Worked example

If a shell redirects `program > out.txt`, the program itself can still write to descriptor 1 as usual. The shell has arranged for descriptor 1 to refer to `out.txt` before the program starts. This demonstrates why redirection is an OS/file-descriptor concept rather than a special feature inside every application.

2.9 Library calls versus system calls

Not every C function that performs I/O maps one-to-one to a kernel transition. `fgetc()` is a C library function operating on a buffered `FILE *`. The library may satisfy multiple character reads from one larger underlying `read()` system call. Buffering reduces expensive transitions and improves throughput.

2.10 Directory entries and pathname lookup

A directory traversal with `readdir()` returns names and implementation-defined directory-entry information. The OS still has to resolve a pathname component by component. For a path such as

`/home/student/report.txt,`

the file system starts from the root, looks up `home`, then `student`, then `report.txt`. Permissions and mount points can affect traversal.

2.11 Defensive systems programming

System interfaces can fail for ordinary reasons: a file does not exist, permissions are insufficient, memory is exhausted, a directory path is invalid, or a process limit is reached. Good systems code therefore treats error handling as part of the algorithm.

Step 1: Check command-line argument counts before indexing `argv`.

Step 2: Check every resource-acquisition result such as `fopen()`, `opendir()`, `fork()`, and memory allocation.

Step 3: Use `perror()` or error codes to preserve the reason for failure.

Step 4: Release resources exactly once on every exit path.

Step 5: Test the return value of the read operation itself rather than predicting EOF in advance.

Worked example

A robust text scan is conceptually:

1. open the file;
2. repeatedly request the next character;
3. process it only if the read succeeded;
4. stop when the read reports end-of-file or error;

5. close the file.

This is why `while ((ch = fgetc(fp)) != EOF)` is preferable to `while (!feof(fp))`.

Check your understanding

1. Why do shells commonly use both `fork()` and `exec()`?
2. What is the difference between a `FILE *` and a POSIX integer file descriptor?
3. Why can parent and child print in either order after `fork()`?

PRACTICAL MILESTONE

Lab 1 of 10

Directories and process creation

Explore a directory and create a child process

Coverage. Chapters 1–2.

Source files. Lab01/Task01.c, Lab01/Task02.c.

Goal. Observe two fundamental OS abstractions: the directory namespace and process identity.

How much is scaffolded?

Both programs are deliberately tiny. Treat them as instruments rather than as products: run one, read its output, change exactly one thing, and run it again. The value is in connecting a line of C to an effect you can see.

Build either with warnings on. They are worth reading, not silencing:

```
gcc -Wall -Wextra Task01.c -o task01 && ./task01
```

Task A — Walking a directory.

A directory is not a list of files. It is a file that happens to contain name-to-inode mappings, and `readdir()` is how you step through them.

```
DIR *dirp = opendir(path);
if (dirp == NULL) {
    perror(path);           /* prints the reason, not just "failed" */
    return EXIT_FAILURE;
}

struct dirent *entry;
while ((entry = readdir(dirp)) != NULL) {
    printf("%s\n", entry->d_name);
}

closedir(dirp);
```

Three details in that fragment are doing real work.

The extra parentheses around the assignment say “I meant to assign here.” Without them the compiler warns, because `=` inside a condition is far more often a typo for `==`.

`perror()` appends the actual reason — No such file or directory, Permission denied — which turns a dead end into a diagnosis.

`closedir()` sits *after* the loop. Putting it inside is the subject of Lab 2, Task B.

Try it yourself

Run it on `.`, then on `/etc`, then on a path that does not exist, then on a directory you lack permission to read. Four different outcomes, all handled by the same six lines.

Now answer: why do `.` and `..` appear in the listing, and is the order alphabetical? Check by running it twice on a directory you have just added a file to.

Task B — Parent and child.

`fork()` is the strangest system call you will meet this semester: it is called once and returns twice.

```
pid_t pid = fork();

if (pid == -1) {
    perror("fork");           /* creation failed */
} else if (pid == 0) {
    printf("child : pid=%ld\n", (long)getpid());
} else {
    printf("parent: pid=%ld, child=%ld\n", (long)getpid(), (long)pid);
    waitpid(pid, &status, 0);
}
```

After `fork()` there are two processes, both sitting on the next line, both holding identical copies of every variable. They are told apart by the return value alone: 0 in the child, the child's PID in the parent, -1 if no child was made.

Common mistake

Use `pid_t`, not `int`. The width of a process ID is platform-defined, and `int` happens to work on Linux today. Cast to `long` for printing, since there is no portable `printf` specifier for `pid_t`.

Try it yourself

Comment out the `waitpid()` call and run the program twenty times in a row:

```
for i in $(seq 20); do ./task02; done
```

Does the parent line always print first? Put `waitpid()` back and repeat. You have just observed a race condition and then removed it.

Then try this: put a `printf()` *before* the `fork()` without a trailing newline, and run the program with output redirected to a file. Count how many times that line appears. Chapter 1 explains why.

Lab deliverable

Submit terminal output for both tasks, plus short answers to:

1. What does `readdir()` return, and how do you tell end-of-directory from an error?
2. What are the three possible return values of `fork()`, and what does each mean?
3. Why can output order vary without `waitpid()`, and why is that not a bug in your program?

PRACTICAL MILESTONE

Lab 2 of 10

File scanning and directory utilities

Build small Unix-style text and directory tools

Coverage. Chapter 2.

Source files. Lab02/Task01.c–Lab02/Task03.c.

Goal. Practise robust read loops, pathname arguments, directory traversal, and line-oriented searching.

How much is scaffolded?

Each of these three programs is built around one mistake that is easy to make, hard to spot, and common in production code. The supplied sources show the correct form; this section explains what the wrong form looks like and why it fails, because recognising the wrong form is the transferable skill.

Task A — Count spaces, and loop correctly.

The natural-sounding version is wrong:

```
/* WRONG --- do not write this */
while (!feof(fp)) {
    ch = fgetc(fp);
    if (ch == ' ') spaces++;
}
```

`feof()` does not predict the end of the file; it reports that a read has *already* failed. So on the last iteration `fgetc()` returns EOF, `feof()` only becomes true afterwards, and the loop body has already run one extra time on a value that is not a character.

Loop on the read itself:

```
int ch, spaces = 0;
while ((ch = fgetc(fp)) != EOF) {
    if (ch == ' ') spaces++;
}
if (ferror(fp)) { perror(filename); } /* real error, not clean EOF */
```

Common mistake

`ch` must be `int`, never `char`.

`fgetc()` returns 256 possible byte values *plus* the distinct sentinel EOF. That needs more range than a `char` has. Squeeze it into a `char` and one real byte becomes indistinguishable from EOF — the loop then ends early on binary input, or on some platforms never ends at all.

Task B — Release resources once, in the right place.

The failure here is placement, not logic:

```
/* WRONG --- closedir() inside the loop */
while ((link = readdir(dp)) != 0) {
    printf("%s", link->d_name);
    closedir(dp); /* handle is dead after the first pass */
}
```

The first iteration releases the directory handle. Every `readdir()` after that reads through a dangling pointer. That is undefined behaviour: it may print rubbish, loop forever, or crash — and which one you get can change between compilers.

Key idea

Acquire, use, release — in that order, once each. The release belongs at the same nesting level as the acquire, never inside a loop that the resource is driving.

The same shape governs `malloc/free`, `fopen/fclose`, and `lock/unlock` in Lab 5.

The corrected program also checks `opendir()` for `NULL` and prints one entry per line, so the output is readable rather than a single run-on string.

Task C — A miniature `grep`.

Read with `fgets()`, strip the newline, search with `strstr()`, print with a line number:

```
while (fgets(line, (int)sizeof line, fp) != NULL) {
    line_number++;

    char *newline = strchr(line, '\n');
    if (newline != NULL) *newline = '\0';

    if (strstr(line, word) != NULL) {
        printf("%s:%ld: %s\n", filename, line_number, line);
        matches++;
    }
}
```

`fgets()` keeps the newline when the line fits in the buffer and omits it when the line is too long. Stripping it conditionally — hence the `NULL` check — is what makes your own formatting predictable.

Try it yourself

Feed it a file whose longest line exceeds `MAX_LINE`. What happens to that line, and does the line counter still tell the truth? Then make the program exit with status 0 when something matched and 1 when nothing did, and confirm with `echo $?`. That is what real `grep` does, and it is what makes it usable in a shell pipeline.

Debugging task

Compile all three with diagnostics enabled and read every line of output:

```
gcc -Wall -Wextra -fsanitize=address,undefined Task02.c -o task02
```

Then deliberately reintroduce one of the three bugs above and run again. AddressSanitizer names the use-after-free in Task B directly, with the line that freed the handle and the line that used it. Record what it printed — learning to read that output is worth more than the exercise itself.

Lab deliverable

Submit source and output for all three tasks, plus a one-page note explaining why loop-on-the-read beats loop-on-`feof()`, and why acquire/use/release discipline matters when the resource is owned by the kernel rather than by your program.

Chapter recap

- Directory streams expose directory entries through `opendir/readdir/closedir`.
- `fork()` creates a second process and returns differently in parent and child.
- File-processing loops should be driven by successful reads, not by a pre-checked EOF flag.
- Correct OS code treats every kernel/library handle as a resource with a clear lifetime.

Stage 2

Shell, Processes, and CPU Management

Shell Scripting and Command-Line Automation

Chapter at a glance

Before you start: You should be able to run commands in a Unix-like terminal.

By the end, you can: Explain the shell's role, use variables, arithmetic, tests, loops and exit status, and write small Bash scripts safely.

3.1 The shell is a command interpreter

A shell such as Bash reads commands, performs expansions, starts programs, connects I/O streams, and exposes programming constructs. It is not the kernel; it is an ordinary user-space program that makes system services convenient to invoke.

3.2 Variables and input

Assignments do not use spaces around `=`. Values are expanded with `$name`. Quoting matters because unquoted expansions can undergo word splitting and wildcard expansion.

```
#!/bin/bash
read -r n
remainder=$(( n % 2 ))
if [ "$remainder" -eq 0 ]; then
    echo "$n is even"
else
    echo "$n is odd"
fi
```

3.3 Arithmetic and conditions

Modern Bash arithmetic expansion `$( ( ... ) )` is usually clearer than legacy `expr`. Numeric tests include `-eq`, `-ne`, `-lt`, `-le`, `-gt`, and `-ge`.

Common mistake

A year divisible by 4 is not always a leap year. The Gregorian rule is: divisible by 400, or divisible by 4 but not by 100. A lab script using only `year % 4` is a useful starting exercise but not the complete calendar rule.

3.4 Loops and factorial

A factorial for non-negative integer n is

$$n! = \prod_{k=1}^n k, \quad 0! = 1.$$

A shell loop can implement the repeated multiplication directly.

```
result=1
for ((i=2; i<=n; i++)); do
    result=$(( result * i ))
done
echo "$result"
```

3.5 Exit status and composition

Every command returns an integer exit status. By convention, 0 means success and non-zero means some form of failure. Shell operators exploit this: `cmd1 && cmd2` runs the second command only if the first succeeds; `cmd1 || cmd2` runs the second only if the first fails.

3.6 Pipes and redirection

The shell can wire one process's standard output to another process's standard input using `|`. Redirection changes the source or destination of a standard stream. These features are built on OS primitives such as file descriptors, pipes, `fork()`, and `exec()`.

Key idea

Shell scripting is operating-systems practice because it exposes process creation, file descriptors, exit status, and command composition without requiring you to implement the low-level plumbing yourself.

3.7 The shell is a process-control language

A shell is not merely a place to type commands. It is a user-space program that parses command lines, expands variables and wildcards, establishes redirections and pipelines, launches processes, and collects exit status. Shell scripting therefore provides a compact way to automate operating-system interactions.

3.8 The execution model of a command line

For a simple external command, a typical shell conceptually performs these steps:

- Step 1:** Parse the command and apply quoting/expansion rules.
- Step 2:** If redirection is requested, prepare the required file descriptors.
- Step 3:** Create a child process.
- Step 4:** In the child, replace the process image with the requested executable.
- Step 5:** In the parent, wait for a foreground command or return immediately for a background job.

Built-in commands such as changing the shell's own current directory cannot always be implemented as ordinary child processes because their effect must change the shell process itself.

3.9 Variables, expansion, and quoting

Shell variables are text values. Expansion happens before the final command is executed, which makes quoting important.

Form	Meaning	Example consequence
<code>\$x</code>	Expand variable	Word splitting/globbering may occur depending on context
<code>"\$x"</code>	Expand while preserving it as one quoted field	Safer for filenames containing spaces
<code>'\$x'</code>	Literal characters; no variable expansion	Prints <code>\$x</code> rather than its value
<code>\$(...)</code>	Arithmetic expansion	Preferred modern integer arithmetic syntax in Bash

Common mistake

Unquoted variables are a common shell-programming bug. A test like `[ $name = admin ]` can break when `name` is empty or contains spaces. Prefer `[ "$name" = "admin" ]`.

3.10 Exit status drives control flow

Every command returns an integer exit status. By convention, zero represents success and nonzero represents some kind of failure or false condition. `if`, `&&`, and `||` are therefore built around command status, not only around numeric expressions.

3.11 Redirection and pipelines

The shell can connect processes without changing their source code.

- `>` replaces standard output with a file.
- `»` appends standard output to a file.
- `<` replaces standard input with a file.
- `|` connects one process's standard output to another process's standard input through a pipe.

A pipeline such as `cat log.txt | grep ERROR | wc -l` illustrates composability: each process performs one transformation while the OS transports bytes between them.

3.12 Conditionals and loops

The lab scripts practice the same control structures used in larger administration scripts: selection and repetition.

```
if (( n % 2 == 0 )); then
    echo "even"
else
    echo "odd"
fi

factorial=1
for ((i = 2; i <= n; i++)); do
    factorial=$((factorial * i))
done
```

3.13 Correctness matters more than syntax

A script can be syntactically valid but algorithmically wrong. The common simplified leap-year rule “divisible by 4” is incomplete. The Gregorian rule is:

$$\text{leap} \iff (y \bmod 400 = 0) \vee ((y \bmod 4 = 0) \wedge (y \bmod 100 \neq 0)).$$

Thus 2000 is a leap year, but 1900 is not.

Worked example

For 1900: divisible by 4, but also divisible by 100 and not by 400, so it is *not* a leap year. For 2000: divisible by 400, so it *is* a leap year.

3.14 Shell scripts as OS laboratory tools

Shell scripts are useful for compiling, executing, and repeatedly testing C programs. A small test harness can run a scheduler with several inputs, capture output, and compare results. This habit turns later OS labs from one-off demonstrations into reproducible experiments.

Check your understanding

1. Why must `cd` normally be a shell built-in?
2. What OS object is created to implement a pipeline?
3. Why should a filename variable usually be quoted when passed to a command?

PRACTICAL MILESTONE

Lab 3 of 10

Bash scripting fundamentals

Use the shell as a lightweight programming environment

Coverage. Chapter 3.

Source files. Four scripts in Lab03.

Goal. Use input, arithmetic, selection, and loops, while handling the edge cases that separate a demo from a tool.

How much is scaffolded?

Each script is four lines of logic wrapped in the input validation that makes it trustworthy. The validation is the lesson. A script that produces a confident wrong answer on bad input is worse than one that refuses to run.

Task	Topic	The edge case that matters
1	Even / odd	Non-numeric input evaluates to 0 and is reported “even”
2	Leap year	Testing only % 4 wrongly accepts 1900 and 2100
3	Factorial	0! must be 1; negative input has no answer at all
4	Swap	Losing a value when the temporary is skipped

Validate before you calculate.

Bash arithmetic silently treats a non-numeric word as zero. So an unvalidated even/odd script reports that abc is an even number, with no warning at all.

```
if ! [[ "$n" =~ ^[+-]?[0-9]+$ ]]; then
    echo "Error: '$n' is not an integer." >&2
    exit 1
fi
```

Two habits appear there and are worth carrying into every script you write. Errors go to `>&2` so they survive a redirect of normal output. Expansions are quoted — `"$n"` — so a value containing a space stays one word instead of becoming two arguments.

The Gregorian rule has three clauses, not one.

A year is a leap year when it is divisible by 400; otherwise not, if divisible by 100; otherwise yes, if divisible by 4.

```
if (( y % 400 == 0 || (y % 4 == 0 && y % 100 != 0) )); then
    echo "$y is a leap year"
else
    echo "$y is not a leap year"
fi
```

The % 100 clause exists because the solar year is slightly under 365.25 days, so a leap day every four years overcorrects. The % 400 clause exists because skipping every century then overcorrects the other way. Testing % 4 alone is not a rounding error — it is wrong for 1900, 2100, 2200, and 2300.

Accumulate into a new variable.

The obvious factorial loop destroys its own input:

```
# WRONG --- n is consumed, so the result cannot name the number it came from,
# and 0! reports 0 because the loop body never runs.
i=`expr $n - 1`
while [ $i -ge 1 ]; do n=`expr $n \* $i`; i=`expr $i - 1`; done
```

```
# RIGHT --- seed at 1 and leave n alone.
product=1
for (( i = 2; i <= n; i++ )); do
    product=$(( product * i ))
done
echo "The factorial of $n is $product"
```

Seeding at 1 handles 0! and 1! with no special case, because the loop simply does not execute when $n < 2$. That is usually a sign you have chosen the right starting value.

Try it yourself

Verify each script against its edge cases before you believe it:

```
for y in 1900 2000 2024 2100; do echo "$y" | bash Task02.sh; done
for n in 0 1 5 -3 abc; do echo "$n" | bash Task03.sh; done
```

Expected: not leap, leap, leap, not leap — then 1, 1, 120, and two refusals.

Now try 21!. Bash integers are 64-bit, so the answer is silently wrong. Find the largest n that still works, and explain how you would detect the overflow rather than printing a wrong number confidently.

Debugging task

Run every script through ShellCheck (`shellcheck Task01.sh`) or paste it into `shellcheck.net`. It flags unquoted expansions, deprecated backticks, and `[ ]` versus `[[ ]]` misuse. Fix what it reports and note which warnings were real bugs rather than style.

Lab deliverable

Submit all four scripts plus a test table: input, expected output, actual output, one row per edge case above. A row where expected and actual differ is a finding, not a failure — explain it.

Chapter recap

- The shell is a user-space command interpreter, not the kernel.
- Bash scripts combine command execution with variables, tests, loops, and arithmetic.
- Quoting and exit-status checking make scripts more reliable.
- Pipes and redirection are convenient syntax over OS-level I/O and process mechanisms.

CPU Scheduling

Chapter at a glance

Before you start: Know the process-state model from Chapter 1, especially the difference between preemption and blocking.

By the end, you can: Compute waiting, turnaround, response, and completion times; trace FCFS, SJF, Priority, and Round Robin on the same workload; and explain what each policy is actually optimising.

4.1 The decision the scheduler makes

Three processes are ready. One CPU is free. Which one runs?

That is the entire question, and the OS answers it thousands of times per second. The **scheduler** applies a policy to choose; the **dispatcher** performs the context switch and hands over the CPU.

What makes this interesting is that there is no correct answer — only trade-offs. Finishing the shortest job first minimises average waiting. Giving everyone a small turn keeps an interactive session responsive. Honouring priorities respects what the user considers important. These goals genuinely conflict, and every policy below picks a side.

4.2 Five quantities, and what each one means to a user

For process i , write A_i for arrival time, B_i for total CPU burst, S_i for the moment it first gets the CPU, and C_i for completion time. Then:

$$\underbrace{T_i = C_i - A_i}_{\text{turnaround}}, \quad \underbrace{W_i = T_i - B_i}_{\text{waiting}}, \quad \underbrace{R_i = S_i - A_i}_{\text{response}}$$

These are not three ways of saying the same thing.

Quantity	What it measures	What the user notices
Turnaround T	Total time in the system, start to finish	“My batch job took 9 seconds”
Waiting W	Time sat in the ready queue doing nothing	Pure overhead — the part a better policy can reduce
Response R	Delay before <i>any</i> progress is visible	“The window froze when I clicked”

Key idea

Waiting time is turnaround minus the work the process actually needed. Since B_i is fixed by the program, minimising average waiting time and minimising average turnaround time are the same problem.

Response time is a different problem, and a policy can be excellent at one and poor at the other.

That is the single most important idea in this chapter.

Worked example

A process arrives at time 2, first receives the CPU at time 5, needs 4 CPU units in total, and finishes at time 11.

$$R = 5 - 2 = 3 \quad (\text{waited 3 units before anything happened})$$

$$T = 11 - 2 = 9 \quad (\text{9 units from arrival to completion})$$

$$W = 9 - 4 = 5 \quad (\text{of those 9, only 4 were useful work})$$

Sanity check: it existed for 9 units, ran for 4, so it must have waited 5. If your W ever exceeds T , or comes out negative, you have made an arithmetic error.

4.3 One workload, four policies

Everything below uses the same three processes, all arriving at time 0. Using one workload throughout is what makes the policies comparable.

Process	Arrival A	Burst B	Priority (lower = more urgent)
P1	0	5	3
P2	0	3	1
P3	0	1	2

4.3.1 First-Come, First-Served

FCFS is non-preemptive and runs processes in arrival order. Here that is P1, P2, P3.

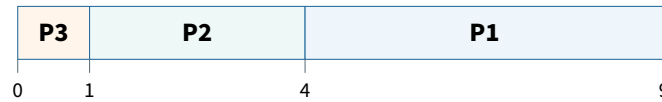


Process	B	C	$T = C - A$	$W = T - B$	$R = S - A$
P1	5	5	5	0	0
P2	3	8	8	5	5
P3	1	9	9	8	8
Average			7.33	4.33	4.33

P3 needs one unit of CPU and waits eight to get it. That is the **convoy effect**: short jobs stuck behind a long one, like cars behind a lorry on a single-lane road.

4.3.2 Shortest Job First

SJF picks the smallest remaining burst: P3, then P2, then P1.



Process	B	C	T	W	R
P3	1	1	1	0	0
P2	3	4	4	1	1
P1	5	9	9	4	4
Average			4.67	1.67	1.67

Average waiting fell from 4.33 to 1.67 — and no work was done faster. The CPU was busy for exactly nine units in both schedules. Only the *order* changed.

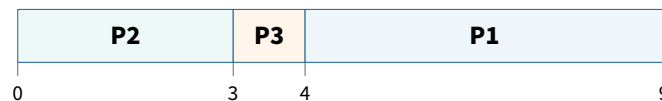
Key idea

For a fixed set of jobs all available at once, SJF provably minimises average waiting time. Running a short job early delays only that short job's successors briefly; running a long job early delays everyone behind it.

The catch is that B_i is the burst the process is *about to* need, which the OS cannot know. Real systems estimate it from recent history, and the estimate is often wrong.

4.3.3 Priority scheduling

Highest priority first. With the convention that a lower number is more urgent, the order is P2, P3, P1.



Averages: $T = 5.33$, $W = 2.33$. Better than FCFS, worse than SJF — because priority happens to correlate with burst length here, but is not chosen to optimise it.

Common mistake

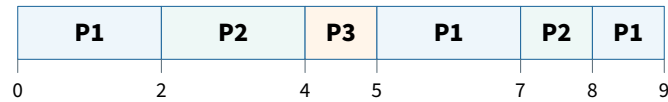
State your priority convention before you compute anything. “Priority 1” means *most* urgent in most textbooks and lab manuals, and *least* urgent in some real systems — Linux `nice` values run from `-20` (most favourable) to `19`.

Half of all lost marks on scheduling questions come from silently assuming the wrong direction. Write the convention at the top of your answer.

Priority scheduling can starve a low-priority process indefinitely if urgent work keeps arriving. The standard remedy is **aging**: gradually raise the priority of anything that has waited a long time, so every process eventually becomes urgent enough to run.

4.3.4 Round Robin

Round Robin is preemptive. Each process gets at most a **time quantum** q , then goes to the back of the queue if unfinished. With $q = 2$ and queue order P1, P2, P3:



Note that P3 finishes early at time 5 despite only getting the CPU third: it needed one unit, which is less than the quantum, so it ran to completion in its first turn.

Process	<i>B</i>	<i>C</i>	<i>T</i>	<i>W</i>	<i>R</i>
P1	5	9	9	4	0
P2	3	8	8	5	2
P3	1	5	5	4	4
Average			7.33	4.33	2.00

4.4 Reading the comparison honestly

Policy	Avg turnaround	Avg waiting	Avg response
FCFS	7.33	4.33	4.33
SJF	4.67	1.67	1.67
Priority	5.33	2.33	2.33
Round Robin ($q = 2$)	7.33	4.33	2.00

Two things in that table are worth more than the numbers themselves.

Round Robin matched FCFS on waiting time and halved response time. Look at the completion times: FCFS produced $\{5, 8, 9\}$ and so did Round Robin. The same three numbers — handed to different processes. FCFS gave the earliest completion to the longest job; RR gave it to the shortest. Round Robin did not make the machine faster. It changed who waits.

SJF won on every column here, and is still not the answer. It needs burst lengths in advance, and it starves long jobs whenever short ones keep arriving. A policy that is optimal under assumptions you cannot satisfy is a benchmark, not a design.

Plain idea

The quantum q is a dial between the two policies you already know.

Set q larger than every burst and no process is ever preempted — Round Robin becomes FCFS. Set q very small and responsiveness improves, but each switch costs real time saving and restoring registers, so an ever-larger share of the CPU goes to switching rather than working.

Real systems land near 10–100 ms, which is short enough to feel instant and long enough that the switch cost stays negligible.

4.5 Drawing a Gantt chart that survives checking

Work in this order, every time:

Step 1: Draw the timeline first, marking every instant the CPU changes hands. Do not compute anything yet.

Step 2: Read the completion time C_i off the chart — the right-hand edge of that process's *last* block.

Step 3: Compute $T_i = C_i - A_i$.

Step 4: Compute $W_i = T_i - B_i$. Never count queue intervals by hand; the subtraction is what stops you double-counting.

Step 5: Read S_i off the chart for R_i — the left edge of that process's *first* block.

Common mistake

The most common Round Robin error is computing waiting time by adding up gaps in the chart. With several preemptions there are several gaps per process, and one is always missed. Use $W = T - B$. It cannot miss a gap, because it never looks at them.

Try it yourself

Redo the same workload with $q = 1$, then with $q = 5$. Confirm that $q = 5$ reproduces the FCFS chart exactly, and explain in one sentence why.

Then change the arrivals to $A_1 = 0$, $A_2 = 1$, $A_3 = 2$ and redo FCFS. The chart now contains idle time if a process has not arrived yet — make sure your C values account for it.

4.6 Scheduling is a queueing problem

At any instant a process may be using the CPU, waiting in the ready queue, or blocked for an event such as I/O. CPU scheduling chooses among *ready* work. It does not make a blocked process runnable; that requires the event it is waiting for to occur.

4.6.1 CPU-bound and I/O-bound behavior

Processes often alternate between **CPU bursts** and **I/O bursts**. An I/O-bound workload tends to have many short CPU bursts separated by waits, whereas a CPU-bound workload tends to use longer CPU bursts. A good general-purpose scheduler must serve both without letting one class dominate the other.

4.7 Preemptive versus non-preemptive scheduling

A **non-preemptive** scheduler allows a running process to keep the CPU until it blocks or terminates. A **preemptive** scheduler may interrupt it and return it to the ready queue. Preemption improves responsiveness and fairness but introduces context-switch overhead and concurrency concerns because a task can be interrupted between operations.

Metric	What it measures	Usually desirable
CPU utilization	Fraction of time CPU does useful work	High
Throughput	Jobs completed per unit time	High
Turnaround time	Arrival to completion	Low
Waiting time	Time spent ready but not running	Low
Response time	Arrival to first CPU service/response	Low, especially interactively
Fairness	Whether work receives a reasonable share	Avoid starvation

4.8 A complete worked scheduling example

Consider four processes, all arriving at time 0:

$$P_1 = 8, \quad P_2 = 4, \quad P_3 = 2, \quad P_4 = 1$$

where the numbers are CPU burst times.

4.8.1 FCFS

Input order gives the Gantt chart

$$P_1 \ 0 \rightarrow 8 \quad P_2 \ 8 \rightarrow 12 \quad P_3 \ 12 \rightarrow 14 \quad P_4 \ 14 \rightarrow 15$$

Waiting times are 0, 8, 12, 14, so

$$\overline{W}_{FCFS} = \frac{0 + 8 + 12 + 14}{4} = 8.5.$$

The three short processes wait behind the long P_1 : this is the convoy effect.

4.8.2 Non-preemptive SJF

Shortest burst first gives P_4, P_3, P_2, P_1 :

$$P_4 \ 0 \rightarrow 1 \quad P_3 \ 1 \rightarrow 3 \quad P_2 \ 3 \rightarrow 7 \quad P_1 \ 7 \rightarrow 15$$

Waiting times are 0, 1, 3, 7, so

$$\overline{W}_{SJF} = 2.75.$$

This dramatic improvement explains why favoring short work is attractive, but in real systems the future burst length is not known exactly.

4.8.3 Round Robin with $q = 2$

The CPU rotates through the ready queue in quanta of at most two time units. The beginning of the trace is

$$P_1(0 \rightarrow 2), P_2(2 \rightarrow 4), P_3(4 \rightarrow 6), P_4(6 \rightarrow 7), \dots$$

P_4 does not finish as early as under SJF, but every process receives service quickly. This is the central response-time benefit of RR.

Key idea

No scheduler is “best” without a workload and objective. SJF is excellent for average waiting time under ideal burst knowledge; RR is attractive for interactive fairness; priority scheduling expresses importance; FCFS has minimal policy complexity.

4.9 How SJF can be approximated

Because the next CPU burst is unknown, a system may estimate it using exponential averaging:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n,$$

where t_n is the measured previous burst and τ_n is the previous prediction. A large α emphasizes recent behavior; a small α smooths over a longer history.

4.10 Priority scheduling, starvation, and aging

If higher-priority jobs continually arrive, a low-priority process may wait indefinitely: **starvation**. **Aging** raises the effective priority of a process as its waiting time grows. This converts “important work first” into a policy that still eventually serves old work.

4.11 Quantum size and context-switch cost

If a time quantum is q and each context switch costs s , making q extremely small can spend a substantial fraction of CPU time switching rather than executing applications. On the other hand, a huge q causes Round Robin to behave like FCFS. Quantum selection is therefore a latency/overhead trade-off.

Worked example

Suppose $q = 2$ ms and a context switch costs 0.1 ms. If every quantum is fully used, roughly $0.1/(2 + 0.1) \approx 4.8\%$ of the CPU interval is switching overhead. With $q = 20$ ms the fraction falls below 0.5%, but an interactive task may wait much longer for its next turn.

4.12 Arrival times change everything

When arrival times differ, a scheduler may become idle before the next job arrives, and SJF can choose only among jobs that have already entered the ready queue. Preemptive shortest-remaining-time scheduling can even replace a currently running job when a shorter one arrives.

Common mistake

Never sort all processes by burst time first when arrival times differ and call the result SJF. The algorithm must make each decision using only processes that are ready at that time.

Check your understanding

1. Which scheduling metric most directly captures how quickly an interactive process first gets CPU time?
2. Why can priority scheduling starve a process, and how does aging help?
3. What does RR approach as the time quantum becomes very large?

PRACTICAL MILESTONE

Lab 4 of 10

CPU scheduling algorithms

Trace and compare four CPU schedulers

Coverage. Chapter 4.

Source files. Four C programs in Lab04.

Goal. Implement and compare FCFS, non-preemptive SJF, priority scheduling, and Round Robin. Use one common workload so comparisons are meaningful. For the supplied programs, arrival times are effectively assumed equal unless you extend the code.

Program	Policy	Verify
Task03.c	FCFS	cumulative waiting time in input order
Task04.c	SJF	ascending burst order and original PID preservation
Task01.c	Priority	priority convention and tie handling
Task02.c	Round Robin	quantum, remaining burst, completion, waiting calculation

Debugging task

For Round Robin, validate waiting time against a hand-drawn Gantt chart. A program can print plausible totals while still updating waiting time incorrectly during intermediate quanta.

Try it yourself

Use bursts $P_1=8$, $P_2=4$, $P_3=2$, $P_4=1$. Compare FCFS and SJF. Then use RR with quantum 2 and report first-response time for every process.

Lab deliverable

Submit source/output for all four algorithms, one common workload, hand-checked Gantt charts, and a comparison table of average waiting and turnaround times.

Check your understanding

1. Why can Round Robin have the same average waiting time as FCFS but much better response time?
2. A process has $T = 12$ and $B = 15$. What went wrong?
3. Why does SJF need an estimate rather than a measurement?
4. What does aging fix, and which policy needs it?

Answers.

1. Waiting time depends only on total time queued, which the same total work distributes similarly. Response time depends on the *first* turn, and RR guarantees every process one within $n \times q$ of

arriving.

2. An arithmetic error. Waiting time would be -3 , and a process cannot exist for less time than it spent running. Recheck C , or check that B is total burst rather than remaining burst.
3. B_i is the burst the process is about to request. It is only measurable after the fact, so schedulers predict it from recent history — typically an exponential average of previous bursts.
4. Starvation. Priority scheduling needs it: without aging, a steady stream of urgent work means a low-priority process may never run at all.

Chapter recap

- Scheduling chooses among ready processes; the dispatcher performs the switch.
- Turnaround, waiting, and response measure different user-visible effects, and no policy optimises all three.
- FCFS is simple and suffers the convoy effect; SJF minimises average waiting but needs unknowable information; priority expresses urgency and needs aging; Round Robin trades throughput for responsiveness.
- Compute waiting as $T - B$, never by summing gaps.
- Every result depends on stated assumptions: arrival times, preemption, quantum, and the priority convention.

Stage 3

Concurrency and Resource Management

Interprocess Communication and Synchronization

Chapter at a glance

Before you start: Know what processes and threads are, and understand that concurrent execution order is not deterministic.

By the end, you can: Explain race conditions, critical sections and semaphores; trace producer-consumer state; and use shared memory with a clear create/attach/use/detach/remove lifecycle.

5.1 Concurrency creates interleavings

Two computations are **concurrent** when their executions overlap in time. On one core this can occur through rapid context switches; on multiple cores instructions can truly execute simultaneously. If shared state is accessed without coordination, different instruction interleavings can produce different outcomes.

5.2 Race conditions and critical sections

A **race condition** occurs when correctness depends on the relative timing of concurrent operations. A **critical section** is code that accesses shared state and must obey a synchronization rule.

A correct critical-section solution aims for:

- **mutual exclusion:** at most one participant is in the protected critical section;
- **progress:** if no one is inside, eligible participants can eventually choose who enters;
- **bounded waiting:** a participant should not be postponed forever.

5.3 Semaphores

A semaphore is an integer synchronization object manipulated through atomic operations conventionally called `wait` (P/down) and `signal` (V/up). A binary semaphore can model a lock; a counting semaphore can represent the number of available resource units.

5.4 Producer-consumer

For a bounded buffer of capacity N , a classic semaphore design uses:

- `mutex` = 1 to protect the buffer data structure;
- `empty` = N for available slots;
- `full` = 0 for occupied slots.

A producer waits for an empty slot and the mutex, inserts an item, then signals the mutex and `full`. A consumer waits for a full slot and the mutex, removes an item, then signals the mutex and `empty`.

Common mistake

A teaching program that implements `wait(s)` as ordinary `-s` and `signal(s)` as `++s` demonstrates the counts but is *not* a real concurrent semaphore. Real synchronization requires atomic operations and blocking/wakeup semantics supplied by the OS or threading runtime.

5.5 Interprocess communication choices

Processes may communicate through pipes, message queues, sockets, signals, or shared memory. **Shared memory** can be very fast because processes access the same memory region after mapping it, but the application must separately solve synchronization.

5.6 System V shared memory lifecycle

The supplied lab uses System V IPC:

1. obtain a key with `ftok()`;
2. create or locate a segment with `shmget()`;
3. attach it with `shmat()`;
4. read/write the mapped bytes;
5. detach with `shmdt()`;
6. remove the segment with `shmctl(..., IPC_RMID, ...)` when appropriate.

Key idea

IPC answers *how data moves between execution contexts*; synchronization answers *when shared data may be safely accessed*. Many systems need both.

5.7 Concurrency creates interleavings

Two threads can be individually correct yet jointly incorrect because their instructions can interleave in many possible orders. A **race condition** occurs when the result depends on an uncontrolled relative timing of concurrent operations on shared state.

Worked example

Suppose shared `x=10` and two threads both execute `x=x+1`. At machine level this is conceptually read, add, write. Both threads can read 10 before either writes, then both write 11. The expected mathematical result 12 is lost. This is a **lost update** race.

5.8 The critical-section problem

A **critical section** is code that accesses shared state requiring controlled concurrency. A classic solution aims for three properties:

Mutual exclusion At most one participant is in the protected critical section when simultaneous access would be unsafe.

Progress If nobody is inside and some processes want to enter, the choice of the next entrant should not be postponed forever by unrelated processes.

Bounded waiting A requester should not be bypassed indefinitely.

5.9 Atomic operations and locks

A lock works only if the operation that acquires ownership is itself atomic. Real processors provide primitives such as test-and-set or compare-and-swap, and OS/runtime libraries build mutexes and semaphores on top of such mechanisms plus scheduler support.

5.9.1 Mutex

A **mutex** expresses ownership of one mutually exclusive region. A thread locks it, enters the critical section, then unlocks it.

5.9.2 Semaphore

A **semaphore** is an integer-like synchronization abstraction manipulated through atomic operations often called wait/P/down and signal/V/up.

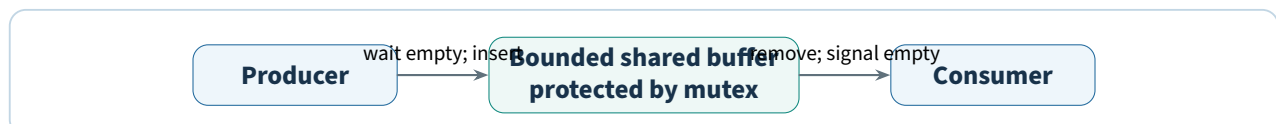
- A **binary semaphore** can act similarly to a lock in simple cases.
- A **counting semaphore** represents multiple units of an available resource.

5.10 Producer-consumer with bounded buffer

For a buffer of capacity N , the canonical semaphore design uses:

$$\text{empty} = N, \quad \text{full} = 0, \quad \text{mutex} = 1.$$

A producer must reserve an empty slot and enter the critical section before inserting an item; a consumer must reserve a full slot and enter the critical section before removing one.



Common mistake

The Lab 5 producer-consumer program changes ordinary integer variables through homemade `wait()` and `signal()` functions. This illustrates semaphore *logic*, but it is not a real concurrent synchronization implementation because those operations are not made atomic and the program does not create concurrent producer/consumer threads.

5.11 Interprocess communication models

Processes can communicate in two broad ways.

Model	Strength	Main correctness concern
Shared memory	Very fast after mapping; processes directly read/write common bytes	Synchronization is explicit and essential
Message passing	Clear send/receive boundaries; easier isolation	Copying/queueing overhead and protocol design
Pipes	Simple byte stream between related processes	Direction, lifetime, blocking, framing
Sockets	Local or network communication through a common abstraction	Protocol, partial I/O, failures, ordering

5.12 System V shared memory in the lab

The Lab 5 shared-memory program demonstrates the lifecycle of a shared segment:

- Step 1:** Obtain/create a segment identifier with `shmget()`.
- Step 2:** Attach it into the process address space with `shmat()`.
- Step 3:** Read or write the mapped bytes.
- Step 4:** Detach with `shmdt()`.
- Step 5:** Remove the segment with `shmctl(..., IPC_RMID, ...)` when no longer needed.

Attaching shared memory does not automatically synchronize readers and writers. If multiple processes modify it concurrently, a separate synchronization scheme is required.

5.13 Safety, liveness, and performance

Concurrency design should be evaluated on three axes:

- **Safety:** nothing bad happens, e.g. no two writers corrupt a protected structure.
- **Liveness:** something good eventually happens, e.g. no permanent deadlock/starvation.
- **Performance:** synchronization does not serialize more work than necessary or cause excessive contention.

Correctness comes first; a very fast data race is still wrong.

Check your understanding

1. Why is `x++` not necessarily atomic?
2. In bounded-buffer synchronization, what do `empty` and `full` count?
3. Why does shared memory require a separate synchronization mechanism?

PRACTICAL MILESTONE

Lab 5 of 10

Synchronization and shared memory

Model producer-consumer state and inspect System V shared memory

Coverage. Chapter 5.

Source files. Lab05/Task01.c, Task02.c.

Goal. Distinguish synchronization state from IPC state.

Task A — Producer-consumer counters.

For the supplied capacity of 3, verify the invariant

$$\text{empty} + \text{full} = 3$$

after every valid produce/consume action.

Common mistake

The lab's integer `wait()` and `signal()` are a sequential simulation. Do not claim that they provide thread-safe atomic semaphore behavior.

Task B — Shared-memory lifecycle.

Trace `ftok` → `shmget` → `shmat` → `access` → `shmdt` → `shmctl(IPC_RMID)`. Use `ipcs -m` before and after removal to observe the kernel object.

Try it yourself

Split shared-memory writing and reading into two separate programs. Run the writer first, keep the segment alive, then run the reader. Add synchronization only if both can access the same bytes concurrently.

Lab deliverable

Submit an invariant trace for producer-consumer, shared-memory terminal output, and a diagram showing the segment's lifecycle and which step creates, maps, detaches, and removes it.

Chapter recap

- Races arise because concurrent instruction order is not fixed.
- Critical-section protocols coordinate access to shared mutable state.
- Semaphores can count resources and enforce ordering, but their operations must be atomic.
- Shared memory is an IPC mechanism; it does not automatically protect itself from races.

Threads, Resource Allocation, and Deadlocks

Chapter at a glance

Before you start: Read Chapter 5 first.

By the end, you can: Create and join POSIX threads, explain mutex protection, state the four deadlock conditions, distinguish prevention/avoidance/detection, and execute the Banker's safety test.

6.1 POSIX threads

The POSIX threads API creates multiple execution streams inside one process. `pthread_create()` starts a thread, `pthread_self()` identifies the calling thread, `pthread_equal()` compares thread identifiers, and `pthread_join()` waits for completion.

```
pthread_t t;
int err = pthread_create(&t, NULL, worker, NULL);
if (err != 0) {
    /* handle error */
}
pthread_join(t, NULL);
```

Common mistake

Thread identifiers are opaque values. Portable code compares them with `pthread_equal()` rather than assuming they are ordinary integers with a specific representation.

6.2 Mutexes

A **mutex** protects a critical section so only one thread owns the lock at a time. The normal lifecycle is initialize, lock, access shared state, unlock, then destroy after all users are finished.

6.3 What deadlock means

A **deadlock** is a state in which a set of processes or threads are permanently blocked because each waits for an event that only another member of the set can cause.

Four Coffman conditions must hold simultaneously for resource deadlock:

1. **Mutual exclusion:** at least one resource cannot be shared concurrently.
2. **Hold and wait:** a process holds resources while requesting more.
3. **No preemption:** resources cannot simply be taken away.
4. **Circular wait:** a cycle of processes each waits for a resource held by the next.

Breaking any one condition prevents this class of deadlock.

6.4 Prevention, avoidance, and detection

Strategy	Core idea	Trade-off
Prevention	Design the protocol so a Coffman condition cannot hold	Can reduce utilization or flexibility
Avoidance	Grant a request only if the resulting state remains safe	Requires future maximum-demand information
Detection/recovery	Allow deadlock, detect it later, then recover	Detection and recovery can be expensive

6.5 Banker's algorithm

Banker's algorithm is a deadlock-avoidance method. For each process and resource type,

$$\text{Need} = \text{Max} - \text{Allocation}.$$

The safety algorithm starts with $\text{Work} = \text{Available}$. It repeatedly finds an unfinished process whose Need vector is component-wise no larger than Work. That process is assumed able to finish, so its Allocation is released into Work. If every process can be placed into such an order, the state is **safe**.

Worked example

Suppose Available is (1, 1), P1 has Need (1, 0) and Allocation (1, 1), while P2 has Need (2, 1) and Allocation (1, 0). P1 can finish first because $(1, 0) \leq (1, 1)$. Work becomes (2, 2), allowing P2 to finish. Therefore a safe sequence is P1, P2.

Common mistake

An **unsafe state** is not identical to an already existing deadlock. Unsafe means the OS can no longer guarantee that every future maximum claim can be satisfied in some order. Deadlock is an actual circular waiting condition.

6.6 Detection versus the supplied exercise

A textbook deadlock-detection algorithm normally uses current outstanding requests rather than each process's declared future maximum. The supplied lab calculates $\text{Need} = \text{Max} - \text{Allocation}$; this is close to a safety/avoidance test. Treat the code as a useful matrix exercise, but keep the conceptual distinction clear.

6.7 Why use threads?

Threads allow multiple execution flows to share one process's address space and resources. Common reasons include responsiveness, parallelism on multicore processors, and natural decomposition of server or pipeline work. Thread creation and communication can also be cheaper than creating fully isolated processes.

6.8 What threads share and what they do not

Threads in one process usually share code, global data, heap, and open files. Each thread requires its own execution state: program counter, registers, scheduling state, and stack. The shared address space makes

communication easy but also creates race conditions.

Resource	Same-process threads	Separate processes
Address space / heap	Shared	Normally isolated
Registers / stack	Private per thread	Private per process/thread
Open-file descriptions	Commonly shared through process resources	Can be inherited/shared intentionally
Failure isolation	Lower	Stronger
Communication	Direct shared memory	IPC required for isolated state

6.9 Pthreads lifecycle

The Lab 6 thread program introduces `pthread_create()`, `pthread_self()`, `pthread_equal()`, and `pthread_join()`. A join is analogous to waiting for another thread to finish and also provides an orderly point for collecting termination state/resources.

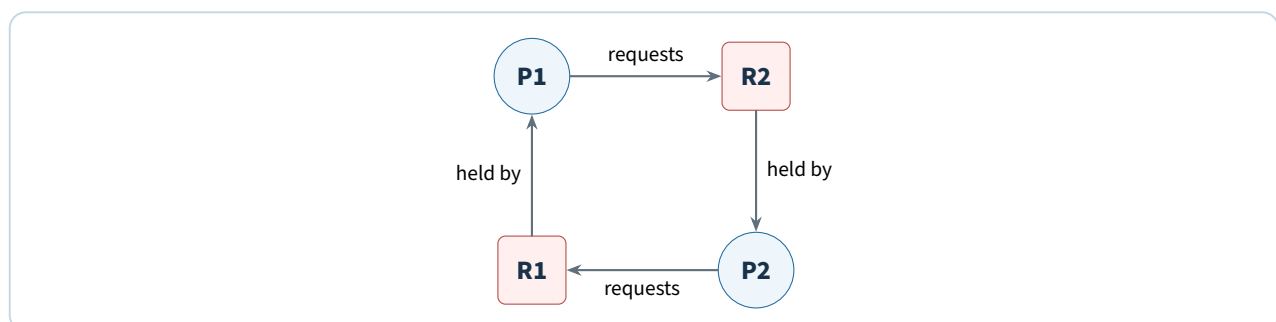
6.10 Deadlock is a circular progress failure

A **deadlock** is a state in which a set of processes/threads cannot make further progress because each is waiting for an event/resource that another member of the set must provide.

The Coffman conditions characterize the classic resource deadlock model. All four must hold for deadlock to be possible:

1. **Mutual exclusion:** at least one resource cannot be shared simultaneously.
2. **Hold and wait:** a process holds resources while requesting additional ones.
3. **No preemption:** resources cannot simply be taken away by the system.
4. **Circular wait:** a cycle of processes exists, each waiting for a resource held by the next.

Breaking any one condition prevents this class of deadlock.



6.11 Four strategies for dealing with deadlock

Prevention Design the protocol so at least one Coffman condition cannot hold, for example impose a global lock/resource ordering to prevent circular wait.

Avoidance Examine requests before granting them and grant only when the system remains in a safe state. Banker's algorithm is the classic example.

Detection and recovery Allow deadlocks to happen, periodically detect them, then recover by terminating

processes, rolling back work, or reclaiming resources if possible.

Ignore In some environments, the system does not run a general deadlock algorithm because the cost/complexity exceeds the expected benefit; applications must avoid problematic cycles.

6.12 Safe state is not the same as current deadlock

A state is **safe** if there exists at least one order in which every process can obtain its remaining maximum needs and finish. An **unsafe** state means no such guaranteed sequence is known under the declared maximum claims. Unsafe does not necessarily mean the processes are already deadlocked at this exact moment; it means the system has lost the safety guarantee.

6.13 Banker's algorithm step by step

For each process i and resource type j :

$$Need_{ij} = Max_{ij} - Allocation_{ij}.$$

Let $Work = Available$. Repeatedly find an unfinished process whose entire Need row is no greater than $Work$. Pretend it finishes, then return its allocated resources:

$$Work \leftarrow Work + Allocation_i.$$

If all processes can be marked finished, the state is safe and the order found is a safe sequence.

Worked example

Suppose one resource type has 10 instances. P1 holds 3 and may need at most 7 total; P2 holds 2 and may need at most 4; P3 holds 2 and may need at most 5. Available is $10 - (3 + 2 + 2) = 3$. Remaining needs are P1=4, P2=2, P3=3. P2 can finish first, returning 2 so $Work$ becomes 5; then P1 or P3 can finish. Therefore a safe sequence exists.

Common mistake

The statement “unsafe = deadlocked” is too strong. Banker's algorithm reasons about whether future completion can be guaranteed under maximum claims. Deadlock detection algorithms answer a different question: which processes are currently unable to complete given present outstanding requests and resources.

6.14 Lock ordering as a practical prevention technique

If every thread acquires multiple locks in one globally agreed order, a circular-wait cycle cannot form. For example, always acquire lock A before B. The rule must be universal; one code path that acquires B before A can reintroduce the cycle.

Check your understanding

1. Name the four Coffman conditions.
2. What is the difference between prevention and avoidance?
3. Why does a safe sequence prove that the declared state is safe?

PRACTICAL MILESTONE

Lab 6 of 10

Threads, Banker's algorithm, and deadlock reasoning

Combine thread lifecycle with resource-allocation safety analysis

Coverage. Chapter 6.

Source files. Three C programs in Lab06.

Goal. Practice POSIX thread creation/joining and matrix-based resource reasoning.

Task A — Banker's safety algorithm.

After input, calculate every Need entry with $\text{Need} = \text{Max} - \text{Allocation}$. Maintain Work and Finish. Record the process selected on each pass and the Work vector after its resources are released.

Task B — Detection terminology.

Compare Task01.c and Task02.c. Both derive Need from Max. Explain why a strict deadlock-detection algorithm is conceptually different from a Banker's safe-state test.

Task C — POSIX threads.

Create two threads, identify them with `pthread_self()`, compare identifiers with `pthread_equal()`, and join both before process exit.

Try it yourself

Remove the busy delay and run the thread program several times. Then add a shared counter without a mutex, followed by a mutex-protected version. Compare results.

Lab deliverable

Submit one safe and one unsafe Banker's input with full Work/Finish traces, a paragraph distinguishing unsafe from deadlocked, and thread output showing successful creation and joining.

Chapter recap

- Threads share a process's resources and therefore need synchronization around shared mutable state.
- Deadlock requires mutual exclusion, hold-and-wait, no preemption, and circular wait.
- Banker's algorithm avoids unsafe allocations by searching for a safe completion sequence.
- Unsafe, deadlocked, and merely blocked are different states.

Stage 4

Memory Management

Contiguous Memory Allocation

Chapter at a glance

Before you start: Know that a process needs memory for code, data, heap, and stack.

By the end, you can: Explain fixed/variable partition allocation, internal and external fragmentation, and trace first-fit, best-fit, and worst-fit policies.

7.1 The allocation problem

A memory manager must map process memory demands onto available physical memory. Early and simplified schemes assign each process one contiguous region. The policy decides which free block or **hole** should satisfy a request.

7.2 Fragmentation

Internal fragmentation is unused space *inside* an allocated region because the allocated unit is larger than the request. **External fragmentation** is free memory split across separated holes so that sufficient total free memory may exist without one large enough contiguous hole.



Separated free holes illustrate external fragmentation.

7.3 First fit

First fit scans holes from the beginning and chooses the first one large enough. It is fast because the search can stop early.

7.4 Best fit

Best fit chooses the smallest hole that is still large enough. It minimizes the leftover size for the current request, but repeated use may create many tiny unusable holes.

7.5 Worst fit

Worst fit chooses the largest available hole, attempting to leave a reasonably large remainder. This does not guarantee lower fragmentation overall.

Policy	Selection rule	Typical observation
First fit	First hole with size $\geq$ request	Low search cost, order-dependent
Best fit	Smallest sufficient hole	Small immediate remainder, can create tiny holes
Worst fit	Largest sufficient hole	Leaves large remainder, may waste large blocks

7.6 Worked comparison

For blocks 100, 500, 200, 300, 600 and requests 212, 417, 112, 426, the three policies choose different placements. To solve such questions consistently:

1. mark every block initially free;
2. process requests in the given order;
3. among currently free blocks, apply exactly one policy rule;
4. compute fragment = block size – request size;
5. mark the chosen block unavailable in the simplified one-file-per-block lab model.

Common mistake

The supplied lab programs model each block as usable by only one file/process. That is a useful partition-allocation simulation, but it is not the same as a full variable-size heap allocator that splits and coalesces holes after every request.

7.7 Why memory management exists

Programs use addresses as though they have their own memory, while physical RAM is finite and shared. Memory management must therefore provide relocation, protection, allocation, and efficient utilization. It also cooperates with virtual memory to let the apparent address space differ from currently resident physical memory.

7.8 Logical versus physical addresses

A CPU instruction typically produces a **logical/virtual address**. Hardware memory-management logic translates it to a **physical address** before RAM is accessed. Even a simple historical base-and-limit scheme illustrates the principle:

$$physical = base + logical, \quad 0 \leq logical < limit.$$

The limit check prevents a process from addressing outside its assigned region.

7.9 Static partitions and dynamic partitions

In fixed partitioning, memory is split into predetermined regions. A process occupies one partition, which is simple but can waste space inside partitions. Dynamic partitioning instead creates variable-sized regions as processes arrive, making hole placement and external fragmentation central problems.

7.10 Internal versus external fragmentation

Internal fragmentation Allocated region is larger than the requested object, so unused bytes are trapped *inside* an allocation.

External fragmentation Enough free memory may exist in total, but it is split among non-contiguous holes that cannot satisfy one large contiguous request.

Worked example

Free holes are 100 KB, 50 KB, and 90 KB. Total free memory is 240 KB. A 150 KB contiguous request still fails because no single hole is large enough. That is external fragmentation.

7.11 Fit algorithms are online decisions

First fit, best fit, and worst fit make a local decision for the current request without knowing the future request sequence. Their behavior therefore depends heavily on allocation order.

Worked example

Free blocks: 100, 500, 200, 300, 600 KB. Requests: 212, 417, 112, 426 KB.

- **First fit:** 212 goes to 500; 417 goes to 600; 112 goes to 200; 426 then fails in the simplified one-block-per-request model.
- **Best fit:** 212 goes to 300; 417 to 500; 112 to 200; 426 to 600.
- **Worst fit:** 212 goes to 600; 417 to 500; 112 to 300; 426 then fails.

This example shows why the lab's policy choice changes the success/failure pattern.

7.12 Splitting, coalescing, and compaction

A real variable-size allocator normally splits a larger free hole when satisfying a smaller request. When adjacent allocations are freed, neighboring holes may be **coalesced**. A system could also perform **compaction**: relocate allocated regions to merge scattered free space. Compaction is expensive and requires relocatable addresses, so modern general-purpose systems rely heavily on paging rather than continuous compaction of entire processes.

7.13 The buddy idea

A common allocation mechanism divides memory into power-of-two block sizes. Larger blocks can be split into equal **buddies**; when both buddies become free, they can merge back into the larger block. This makes coalescing efficient but introduces internal fragmentation because requests are rounded to supported block sizes.

Beyond the core

Kernel memory allocators often combine several techniques. Page-level allocators manage physical frames, while slab/slub-style allocators efficiently reuse caches of frequently allocated kernel objects. The important lesson is that “memory allocation” is not one universal algorithm; different layers optimize for different object sizes and lifetimes.

7.14 Allocation policy versus the Lab 7 simulation

The Lab 7 programs correctly teach the selection rules, but their `bf []` arrays mark an entire input block unavailable after one assignment. This is closer to allocating from a set of fixed candidate partitions than to a fully splitting dynamic-hole allocator. When interpreting results, state this model explicitly.

Check your understanding

1. Explain internal and external fragmentation using one sentence each.
2. Why can compaction address external fragmentation but not be free of cost?
3. What is the local decision made by best fit?

PRACTICAL MILESTONE

Lab 7 of 10

First fit, best fit, and worst fit

Compare contiguous allocation policies on identical requests

Coverage. Chapter 7.

Source files. Three C programs in Lab07.

Goal. See how a local placement rule changes fragmentation and future choices.

Use the same block and request sequence for all policies. For each request record: selected block, block size, request size, fragment, and whether allocation failed.

Worked example

If free blocks are 120, 500, 250 and the request is 200, first fit chooses 500, best fit chooses 250, and worst fit chooses 500. The immediate fragment is 300 for first/worst fit and 50 for best fit.

Try it yourself

Find a request sequence for which best fit allocates an early request efficiently but causes a later request to fail while first fit succeeds. Explain the state left after each decision.

Lab deliverable

Submit outputs for all three algorithms using one common dataset and a comparison table of total reported fragmentation and number of failed requests.

Chapter recap

- Contiguous allocation assigns each request one continuous region.
- Internal fragmentation wastes space inside an allocation; external fragmentation scatters free space between allocations.
- First, best, and worst fit differ only in the hole-selection policy, but that choice changes future memory state.

Virtual Memory and Page Replacement

Chapter at a glance

Before you start: Understand contiguous allocation from Chapter 7.

By the end, you can: Explain pages and frames, translate a virtual page through a page table, define page faults, and trace FIFO, LRU, and LFU replacement.

8.1 Why virtual memory changes the problem

Paging divides virtual memory into fixed-size **pages** and physical memory into equal-size **frames**. A page table records which frame currently contains each resident virtual page. Because pages need not occupy adjacent frames, paging removes external fragmentation from physical allocation, though the last page of an object can still create internal fragmentation.

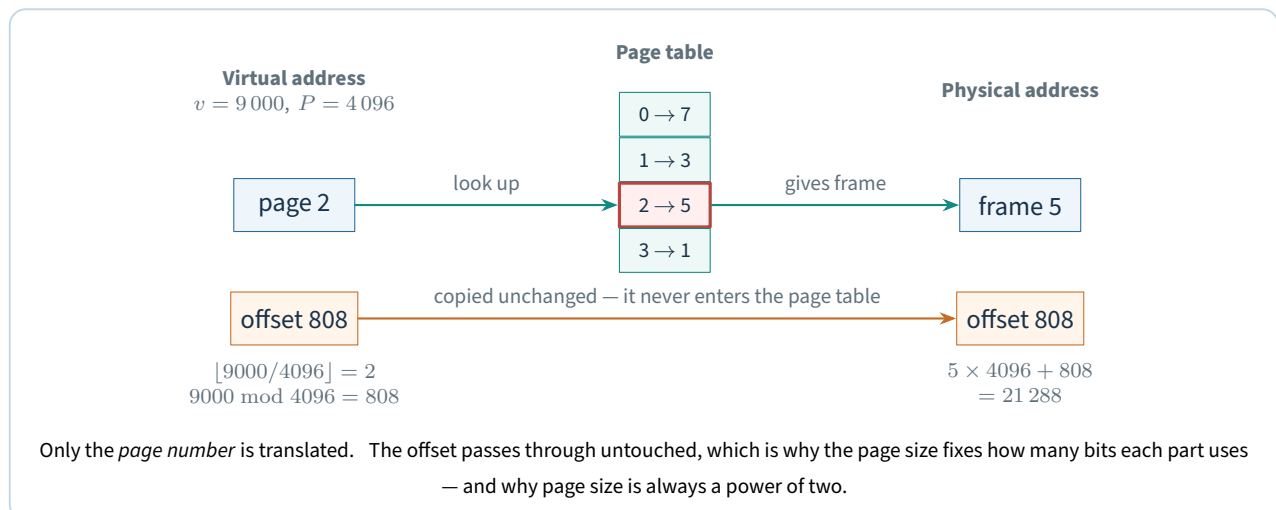
8.2 Address translation

If the page size is P , a virtual address v can be decomposed into

$$\text{page} = \left\lfloor \frac{v}{P} \right\rfloor, \quad \text{offset} = v \bmod P.$$

If the page table maps that page to frame f , then

$$\text{physical address} = fP + \text{offset}.$$



Key idea

Translation replaces the page number and copies the offset. If a virtual address is contiguous inside one page, its physical address is contiguous too; across a page boundary it need not be, because the next page may live in any frame.

8.3 Page faults

A **page fault** occurs when a referenced virtual page is not currently resident in a usable physical frame. The OS must locate the page on backing storage (or otherwise materialize it), obtain a frame, update mapping metadata, and restart/resume the faulting instruction as appropriate.

8.4 Replacement policies

When no free frame exists, the OS needs a victim page.

Policy	Victim rule	State required
FIFO	Page resident for the longest time	Arrival/load order
LRU	Page whose most recent use is farthest in the past	Recency information
LFU	Page with the smallest reference count	Frequency counts plus a tie-breaker

8.5 FIFO

FIFO treats loaded pages like a queue. It is easy to implement but does not use actual access behavior. FIFO can exhibit **Belady's anomaly**: for some reference strings, adding more frames can increase the number of faults.

8.6 LRU

LRU approximates the principle of temporal locality: a page used recently is likely to be used again soon. Exact LRU can be expensive, so real systems often use hardware-assisted approximations.

8.7 LFU

LFU retains frequently referenced pages. Pure cumulative counts can preserve stale historical popularity, so practical frequency-based schemes often age counts or use bounded windows.

Worked example

With three frames and reference string 1, 2, 3, 1, 4: the first three references fault and fill the frames. The second reference to 1 is a hit. On reference 4, FIFO evicts 1 if 1 was loaded earliest, whereas LRU evicts 2 because 2 is the least recently used at that moment.

8.8 Mutex exercise in this lab set

The first program in Lab 8 is actually a mutex synchronization demonstration rather than an address-translation program, despite its opening comment. This is a good reminder that source comments are documentation, not truth: read the executable logic and verify what it does.

Key idea

A page-replacement algorithm is a prediction policy: it tries to choose the resident page whose removal will hurt future execution least.

8.9 Virtual memory decouples address space from RAM

Virtual memory lets a process use a large, private-looking address space even though physical RAM is shared and may contain only a subset of that process's pages at a particular moment. The abstraction improves protection, relocation, sharing, and memory utilization.

8.10 Page-table entries contain more than a frame number

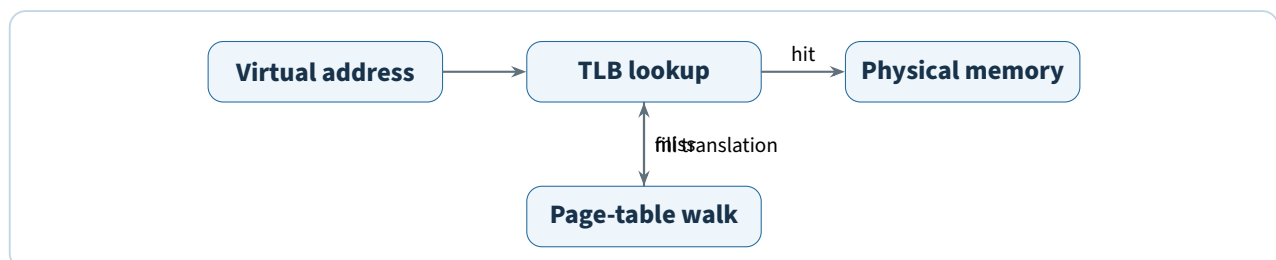
A page-table entry can conceptually include:

- the physical frame number if resident;
- a present/valid bit;
- permission bits such as read/write/execute and user/kernel access;
- an accessed/reference bit useful for replacement approximations;
- a dirty/modified bit indicating whether the page must be written back before eviction.

The exact hardware format varies by architecture.

8.11 The TLB makes translation practical

Consulting an in-memory page table for every memory reference would add large overhead. A **translation lookaside buffer (TLB)** is a small associative cache of recent virtual-page to physical-frame translations. On a TLB hit, translation is fast; on a TLB miss, hardware/software walks the page table and can then cache the result.



8.12 Demand paging and the page-fault path

With demand paging, a page is brought into physical memory when it is first needed rather than loading the entire process in advance. A fault commonly involves:

- Step 1:** Hardware detects that the translation is not currently usable and traps to the kernel.
- Step 2:** The kernel verifies whether the reference is legal.
- Step 3:** It finds a free frame or selects a victim frame.
- Step 4:** If the victim is dirty, its contents may need to be written back.
- Step 5:** The required page is read/materialized into the frame.

Step 6: Page-table/TLB state is updated.

Step 7: The faulting instruction resumes.

Disk/storage access is many orders of magnitude slower than a normal RAM reference, so page-fault rate has a major effect on performance.

8.13 A detailed replacement trace

Take reference string

1, 2, 3, 1, 4, 5, 2, 1, 2, 3, 4, 5

with three frames.

For FIFO, maintain the load order. For LRU, update recency on *every hit and fault*. The algorithms can choose different victims because FIFO asks “who arrived earliest?” while LRU asks “who was used least recently?”.

Key idea

A page hit is not a no-op for all algorithms. In LRU it changes recency state; in LFU it changes the frequency count. A trace that updates state only on faults is usually wrong for these policies.

8.14 Optimal replacement as a benchmark

The theoretical **OPT** algorithm evicts the page whose next use is farthest in the future. It is impossible to implement perfectly online because future references are unknown, but it gives a lower-bound benchmark for a known trace. LRU is motivated by the hope that recent past usage predicts near-future usage.

8.15 Belady’s anomaly and stack algorithms

FIFO can exhibit Belady’s anomaly: increasing the number of frames can increase page faults for some strings. LRU and OPT are **stack algorithms**; the set of pages resident with n frames is contained in the set resident with $n + 1$ frames under the same reference history, so they do not show this anomaly.

8.16 Locality, working set, and thrashing

Programs exhibit **locality**: during a phase, they tend to reuse a relatively small set of code/data pages. The **working set** approximates the pages actively needed in a recent time window. If a process receives too few frames for its active working set, it may fault repeatedly. When the system spends most of its time paging rather than executing useful instructions, it is **thrashing**.

Analogy

A student working at a desk needs a small active set of books. If the desk is too small, the student constantly walks to the shelf to swap books. A larger library does not help if the desk cannot hold the books needed for the current task. RAM frames are the desk; secondary storage is the shelf.

8.17 Page size trade-offs

Larger pages reduce page-table entry counts and can improve sequential I/O efficiency, but they can waste more space through internal fragmentation and move more unused data on each fault. Smaller pages improve granularity but require more mapping metadata and can increase TLB pressure.

Check your understanding

1. What is the purpose of the TLB?
2. Why must LRU update metadata on a page hit?
3. What does thrashing mean in performance terms?

PRACTICAL MILESTONE

Lab 8 of 10

Mutexes and page-replacement policies

Protect a shared critical section and trace FIFO, LRU, and LFU

Coverage. Chapter 8.

Source files. Four C programs in Lab08.

Goal. Connect synchronization with virtual-memory policy tracing.

Task A — Mutex.

The first source creates two threads and holds a pthread mutex around the entire simulated job. Explain why this serializes the jobs even though two threads exist.

Task B — FIFO.

Maintain a circular insertion pointer. A hit does not advance the pointer; a fault replaces the frame at the current pointer and advances it modulo the frame count.

Task C — LRU.

On replacement, identify the resident page whose last reference is farthest back in the reference string.

Task D — LFU.

Track frequency and state the tie-breaker explicitly. The supplied code uses recency information to break certain equal-frequency cases.

Debugging task

The opening comment of `Task01.c` describes address translation, but the executable code demonstrates a pthread mutex. Document what the code actually does rather than repeating the stale comment.

Lab deliverable

Submit a frame trace for one shared reference string under FIFO, LRU, and LFU, with hit/fault totals and a short explanation of every replacement victim.

Chapter recap

- Paging separates virtual pages from physical frames through a page table.
- A page fault is an exceptional control transfer caused by a non-resident or otherwise invalid mapping.
- FIFO uses age, LRU uses recency, and LFU uses frequency to choose victims.
- The correctness of a trace depends on frame count, reference order, hit handling, and tie-breaking rules.

Stage 5

File Systems and Storage

Directories and File-System Namespace

Chapter at a glance

Before you start: Know how `opendir/readdir` expose directory entries.

By the end, you can: Explain directory trees, absolute and relative pathnames, distinguish file names from file metadata, and connect the Lab 9 visualizations to real file-system structures.

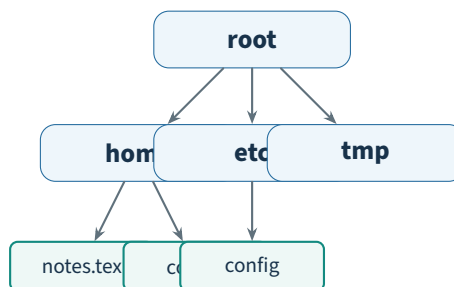
9.1 Why directories exist

A file system needs a namespace that maps human-readable pathnames to file-system objects. A directory stores associations between names and underlying objects or metadata references. Hierarchical directories allow organization, delegation, and scalable lookup.

9.2 Common directory organizations

Textbooks often introduce:

- **single-level:** every file shares one directory;
- **two-level:** each user gets a separate directory;
- **tree-structured:** directories may contain files and subdirectories;
- **acyclic/general graph:** links allow more complex sharing, requiring cycle/reference management.



9.3 Absolute and relative paths

An **absolute path** begins from the root of the namespace. A **relative path** begins from the process's current working directory. Special entries such as `.` and `..` conventionally mean current and parent directory.

9.4 Name versus metadata

A filename is not the complete file. Real file systems maintain metadata such as object type, owner, permissions, timestamps, size, and pointers to data blocks. In Unix-like systems, directory entries associate names with inode numbers; the inode stores most metadata and block mapping information.

9.5 What Lab 9 visualizes

The supplied OpenGL programs model a root directory with file children and a more general interactive tree whose nodes may represent directories or files. The graphics are not implementing an on-disk file system; they visualize the *namespace structure* and parent-child relationships.

Common mistake

A GUI tree node should not be confused with the physical location of file data. Directory hierarchy is a logical namespace; data blocks can be placed elsewhere on storage.

9.6 Directory operations

Important logical operations include search, create, delete, list, rename, link/unlink, and traverse. A well-designed file system must implement these while preserving consistency if power fails or operations race.

9.7 A filesystem separates names, metadata, and data

A useful mental model is to separate three things:

1. **Name:** a directory entry such as `report.txt`.
2. **Metadata object:** ownership, permissions, timestamps, size, and block mapping (for example, an inode in Unix-like designs).
3. **Data blocks:** the bytes that belong to the file.

This separation explains why renaming a file can be cheap: the operation may only change a directory entry while leaving file data where it already is.

9.8 Pathname resolution is repeated directory lookup

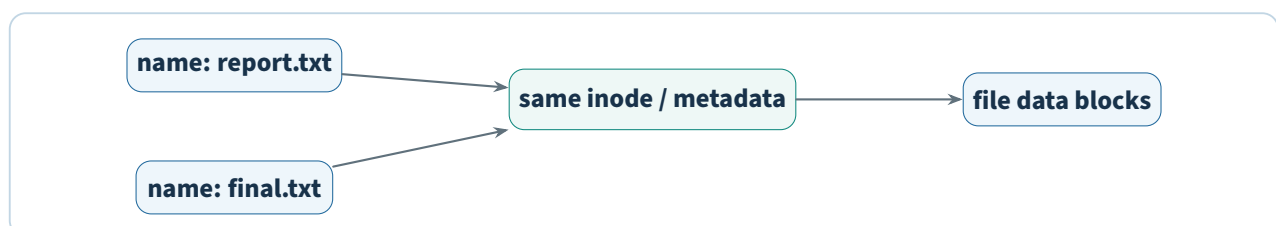
Resolving `/a/b/c.txt` means starting from the root, looking up `a`, then looking up `b` inside that directory, then `c.txt`. For a relative path, the process starts from its current working directory instead.

Every directory in the traversal path requires appropriate search/execute permission under Unix-style protection. This is why having permission on the final file alone may not be enough if an ancestor directory blocks traversal.

9.9 Hard links and symbolic links

Hard link Another directory entry referring to the same underlying filesystem object/inode. Removing one name does not delete the data while other hard links remain.

Symbolic link A separate small file whose contents identify another pathname. Resolution follows that pathname, so a symlink can become dangling if the target disappears.



9.10 Directory structure evolution

Single-level and two-level directory models are useful historically because they expose why hierarchy became necessary. Tree-structured directories scale naming naturally, while links make the namespace graph-like. Once cycles are possible, traversal utilities must avoid infinite recursion by tracking visited objects or applying other rules.

9.11 Directory operations and consistency

Create, unlink, rename, and link operations update metadata structures. A crash halfway through such an update can leave inconsistent state unless the filesystem uses ordering, journaling, copy-on-write, or recovery mechanisms. This is why directory operations are an OS consistency topic, not just a UI topic.

9.12 Caching directory and metadata lookups

Repeated pathname resolution would be expensive if every component required storage I/O. Operating systems cache recently used directory entries and metadata in memory. Cache hits make operations such as opening commonly used libraries dramatically faster.

9.13 Connecting Lab 9 graphics to the real abstraction

The OpenGL tree visualizes parent-child namespace relationships. To interpret it as an OS model:

- a displayed directory node represents a namespace container, not a physically contiguous disk region;
- an edge represents name containment/lookup relationship, not necessarily a physical pointer drawn exactly as on disk;
- expand/collapse is a UI operation; it does not create or delete filesystem objects;
- coordinates and animations are presentation state unrelated to file metadata.

Check your understanding

1. Why can two hard-link names refer to the same file contents?
2. What is the starting point for an absolute path versus a relative path?
3. Why does the directory tree not imply that file data is physically stored in the same tree shape?

PRACTICAL MILESTONE

Lab 9 of 10

Directory-structure visualization

Visualize flat and hierarchical directory organizations

Coverage. Chapter 9.

Source files. Two OpenGL/freeglut programs in Lab09.

Goal. Use graphics to make file-system namespace relationships concrete.

Task A — Root with file children.

The first program draws a root directory and a set of named child file nodes. Identify which parts correspond to namespace data (names, parent-child relations) and which parts are purely presentation (coordinates, colors, shapes).

Task B — Interactive tree.

The second program recursively creates directory/file nodes, lays out children, supports visibility/collapse state, and performs mouse picking. Relate recursion in the program to recursion in pathname traversal.

Try it yourself

Add one metadata field such as file size or permissions and display it without changing the logical parent-child tree. This reinforces the distinction between object metadata and directory hierarchy.

Lab deliverable

Submit screenshots of both visualizations and a labeled diagram distinguishing logical directory structure from physical disk-block placement.

Chapter recap

- Directories provide a pathname namespace over file-system objects.
- Tree structures make nested organization natural, while links can introduce graph-like sharing.
- Pathnames describe logical location; storage blocks need not mirror the directory tree physically.

File Organization and Disk-Block Allocation

Chapter at a glance

Before you start: Read Chapter 9 first.

By the end, you can: Explain text versus binary records, sequential search, indexed allocation, and compare contiguous, linked, and indexed file allocation.

10.1 Records inside files

Applications often impose a record structure on a byte stream. The Lab 10 programs store student or employee records and then display/search them. Two broad encodings appear:

- **text format:** fields are converted to characters with functions such as `fprintf()` and parsed with `fscanf()`;
- **binary format:** an in-memory structure is written as raw bytes with `fwrite()` and read with `fread()`.

Format	Strength	Caution
Text	Human-readable, easier interoperability	Parsing overhead and larger representation
Raw binary struct	Simple and compact within one build environment	Padding, endianness, type sizes, and ABI make it non-portable as a durable interchange format

10.2 Sequential search

A simple search rewinds the file and scans one record at a time until a key matches. Cost is $O(n)$ records in the worst case. Larger systems add indexing structures to reduce lookup cost.

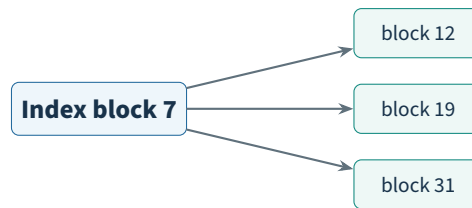
10.3 File allocation on storage

The OS/file system must map a file's logical blocks to physical storage blocks.

- **Contiguous allocation:** file occupies consecutive blocks. Fast sequential/random access, but growth and external fragmentation are concerns.
- **Linked allocation:** each block points to the next. Easy growth and no external fragmentation, but random access is inefficient and pointer integrity matters.
- **Indexed allocation:** a dedicated index block stores pointers to the file's data blocks. Supports direct access but consumes index space.

10.4 Indexed allocation in the supplied lab

Lab 10 task 3 uses an integer array to model whether a disk block is allocated. The user chooses an index block and a set of data blocks. The simulation rejects a request if any referenced block is already marked allocated.



Common mistake

Using `goto START` after discovering a conflict can leave earlier temporary choices or the index-block mark in an inconsistent state unless rollback is carefully implemented. Transaction-like allocation should validate the whole request before committing state changes.

10.5 Durability and consistency

Real file systems must survive partial writes and crashes. Journaling, copy-on-write, checksums, write ordering, and recovery tools are examples of mechanisms used to keep metadata and file content consistent.

10.6 From a byte stream to application records

At the OS interface, a regular file is fundamentally a sequence of bytes plus metadata. “Student records” or “employee records” are an application-level interpretation layered on top. The application decides field layout, delimiters, binary encoding, and how records are searched.

10.7 Access methods

Sequential access Process bytes/records in order. Excellent for logs, streams, and full scans.

Direct/random access Jump to a known byte or fixed-size record offset. Requires predictable mapping from logical position to file location.

Indexed access Maintain an auxiliary mapping from key to record/block location. This can dramatically reduce lookup work at the cost of extra metadata and update complexity.

Worked example

If fixed-size records are exactly 64 bytes and the file contains no header, record i begins at byte offset $64i$. A seek can reach it in constant arithmetic time. In a variable-length text file, record i cannot generally be located this way without an index because earlier records have different lengths.

10.8 Logical blocks versus physical blocks

Filesystems usually reason in blocks. The application sees logical file block 0, 1, 2, ... while filesystem metadata maps those logical positions to physical storage locations. Allocation methods differ mainly in how that mapping is represented.

10.8.1 Contiguous allocation

Store a file in consecutive blocks. Metadata can be as small as start block plus length. Sequential and random access are excellent, but growing the file can be difficult if the following blocks are occupied.

10.8.2 Linked allocation

Each block contains a pointer/reference to the next. A file can grow by allocating any free block, avoiding external fragmentation in the classic sense. However, reaching logical block k may require following the chain from the beginning, and pointer corruption can lose the remainder of the chain.

10.8.3 Indexed allocation

Store block pointers in a dedicated index structure. This supports direct lookup of logical blocks and non-contiguous placement. Large files may require multi-level indexing or multiple index blocks.

Method	Random access	Growth	Main overhead/risk
Contiguous	Excellent	Difficult if adjacent space unavailable	External fragmentation / relocation
Linked	Poor without extra indexing	Easy	Pointer overhead and chain traversal
Indexed	Good	Flexible	Index-block space and metadata complexity

10.9 Extents: a practical compromise

Many modern filesystems map files using **extents**: each extent describes a run of consecutive blocks. A large file can consist of several extents. This keeps metadata smaller than one pointer per block while allowing growth without requiring the entire file to remain one contiguous run.

10.10 Managing free space

The filesystem also needs to know which blocks are unused. Common approaches include bitmaps/bit vectors, free lists, grouping, and counting runs of free blocks. Allocation performance depends on how quickly suitable free regions can be found.

10.11 Crash consistency and journaling

A high-level operation such as “append data and rename the file” can require several low-level metadata writes. If power fails after only some writes reach storage, metadata may become inconsistent. **Journaling** records enough intended metadata change in a log so recovery can replay or discard incomplete operations according to the filesystem’s protocol. Copy-on-write filesystems instead write new versions and then atomically switch references when possible.

Key idea

Durability is not the same as successful execution of `fwrite()`. Library buffering, kernel page cache, device caches, and filesystem ordering all lie between an application call and persistent storage. Systems that require strong durability must use the appropriate flush/synchronization primitives and understand their guarantees.

10.12 Binary-structure portability

Writing a C struct directly with `fwrite()` is convenient for a lab, but a durable file format should define field sizes and byte order explicitly. Compilers may insert padding between fields; architectures can differ in endianness and type width. A portable binary format serializes values deliberately rather than persisting an arbitrary in-memory struct layout.

10.13 Transactional thinking for allocation

The indexed-allocation lab should conceptually perform *validate then commit*:

- Step 1:** Validate that the requested index block is in range and free.
- Step 2:** Validate every requested data block and reject duplicates.
- Step 3:** Only after the entire request is known to be valid, mark all involved blocks allocated.

This avoids partial state changes when a later validation step fails.

Check your understanding

1. Why is indexed allocation better suited to direct access than linked allocation?
2. What portability issue arises from writing raw C structs to disk?
3. What problem does a filesystem journal help solve?

PRACTICAL MILESTONE

Lab 10 of 10

File records and indexed allocation

Store application records and simulate indexed file allocation

Coverage. Chapter 10.

Source files. Three C programs in Lab10.

Goal. Separate application-level record format from file-system-level block allocation.

Task A — Binary student records.

Write structures with `fwrite()`, rewind the stream for display/search, and use `fread()` until it returns fewer than one complete record.

Task B — Text employee records.

Write with `fprintf()` and parse with `fscanf()`. Compare readability and portability with raw structure bytes.

Task C — Indexed allocation simulation.

Model an index block pointing to several allocated data blocks. Validate that all requested blocks are free before committing the allocation.

Debugging task

Improve the indexed-allocation program so a failed request does not leak the newly marked index block. Implement a two-phase pattern: validate every block first, then mark all blocks allocated only if validation succeeds.

Lab deliverable

Submit binary/text record examples, a search demonstration, and an indexed-allocation trace that includes at least one rejected overlapping request and one successful allocation.

Chapter recap

- Text and binary files encode records differently and have different portability trade-offs.
- Sequential search is simple but scales linearly with record count.
- Contiguous, linked, and indexed allocation trade placement simplicity, access speed, growth flexibility, and metadata overhead.

Revision Map and Capstone Practice

The five exam-solving habits

Most Operating Systems questions become manageable if you use a repeatable method:

1. **Name the state.** Write the queue, matrices, frame contents, free blocks, or directory nodes before applying a policy.
2. **Name the event.** Arrival, dispatch, quantum expiry, resource request, memory request, page reference, or file-block allocation.
3. **Name the policy.** FCFS, SJF, Priority, RR, Banker's safety rule, first/best/worst fit, FIFO/LRU/LFU.
4. **Update exactly once.** Change the state according to the event and policy; do not mentally skip intermediate states.
5. **Compute metrics last.** Waiting time, turnaround, page faults, fragmentation, or safe sequence should follow from the completed trace.

Question type	State to draw	Final evidence
CPU scheduling	Ready queue + Gantt chart	Completion, turnaround, waiting, response
Banker's algorithm	Allocation, Max, Need, Work, Finish	Safe sequence or reason none exists
Memory fit	Free-block table	Chosen block + fragment for each request
Page replacement	Frame table after every reference	Hit/fault count and victim sequence
File allocation	Free/used block map	Valid block mapping with conflicts rejected

Capstone practical

Build one command-line simulator that lets a user choose between CPU scheduling, memory allocation, or page replacement. Separate *policy* from *state*: for example, one common frame-state representation can be reused with FIFO, LRU, and LFU victim-selection functions.

Mini-project

Minimum features.

- Scheduling: FCFS, SJF, Priority, RR with a printed Gantt trace and average waiting/turnaround time.
- Memory: first, best, worst fit with fragmentation results.
- Virtual memory: FIFO, LRU, LFU with per-reference frames, hits, and faults.
- Input validation and deterministic tie-breaking rules.
- A short README explaining assumptions and how to compile/run the program.

Decision memo

When two algorithms give different answers, do not ask only “which is better?” Ask which objective matters: responsiveness, throughput, fairness, memory utilization, implementation overhead, locality, or predictable behavior.

Final checklist**Check your understanding**

1. Can you explain the parent/child return values of `fork()` without running code?
2. Can you derive waiting and turnaround time from a Gantt chart?
3. Can you distinguish a race condition from deadlock?
4. Can you compute Need and search for a Banker’s safe sequence?
5. Can you distinguish internal from external fragmentation?
6. Can you trace FIFO, LRU, and LFU one reference at a time?
7. Can you explain why directory hierarchy is not the same thing as physical disk layout?
8. Can you compare text/binary records and contiguous/linked/indexed allocation?